

生成式 AI 学习笔记

Author: 张哲源

Email:

Date: 2026-02-19

1 变分自编码器 (Variational Autoencoder, VAE)

表 1-1 符号说明

符号	含义
$\mathcal{D}$	数据集, $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$
$\mathbf{x}^{(i)}$	第 i 个数据样本, 维度为 d
$\mathbf{z}$	瓶颈层学习到的压缩隐变量 (Latent Code)
$g_{\phi}(\cdot)$	编码器函数, 参数为 ϕ
$f_{\theta}(\cdot)$	解码器函数, 参数为 θ
$q_{\phi}(\mathbf{z} \mathbf{x})$	推断网络 (近似后验), 概率编码器
$p_{\theta}(\mathbf{x} \mathbf{z})$	生成网络 (似然), 概率解码器

1.1 前置知识：自编码器及其变体 (Autoencoders)

Definition 1.1 (自动编码器). 自动编码器 (Autoencoder, AE) 是一种无监督学习模型, 其核心目标是学习一个恒等函数 (Identity Function), 即输出 $\mathbf{x}' \approx \mathbf{x}$ 。它通过一个具有窄瓶颈层 (Bottleneck) 的神经网络将高维输入压缩为低维编码 (降维), 再从该编码重构出原始输入。

1.1.1 基本自动编码器 (Autoencoder)

自动编码器通常包含两个部分:

- 编码器 (Encoder)** $g_{\phi}(\cdot)$: 将高维输入 $\mathbf{x}$ 映射为低维隐变量 $\mathbf{z} = g_{\phi}(\mathbf{x})$ 。
- 解码器 (Decoder)** $f_{\theta}(\cdot)$: 将隐变量 $\mathbf{z}$ 映射回重构数据 $\mathbf{x}' = f_{\theta}(\mathbf{z})$ 。

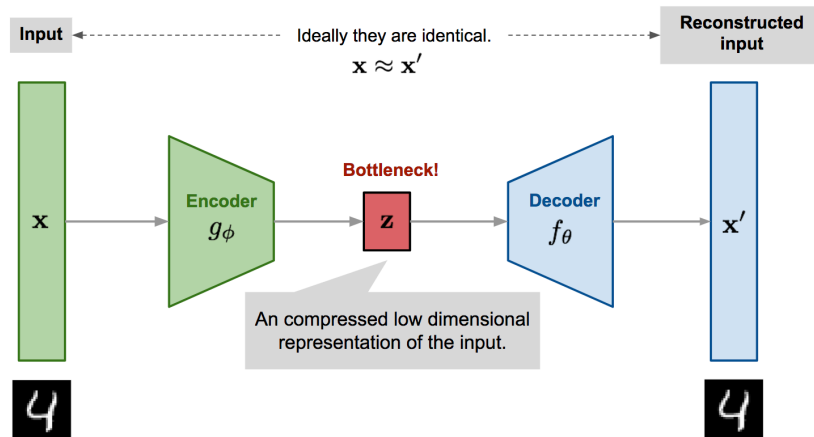


图 1-1 自动编码器模型架构示意图

训练目标是 최소화 重构误差，例如均方误差 (MSE):

$$L_{AE}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - f_{\theta}(g_{\phi}(\mathbf{x}^{(i)})))^2 \quad (1)$$

1.1.2 去噪自动编码器 (Denoising Autoencoder, DAE)

为了避免过拟合（即单纯地记忆输入数据），去噪自动编码器引入了“破坏”机制。

Note 1.1 (核心理念). DAE 在输入 $\mathbf{x}$ 中主动加入噪声得到 $\tilde{\mathbf{x}}$ （例如将部分像素置零，类似 Dropout），然后训练模型从损坏的 $\tilde{\mathbf{x}}$ 中恢复出原始的无噪数据 $\mathbf{x}$ 。这迫使模型学习数据维度间的依赖关系，从而提取更鲁棒的特征。

损失函数调整为：

$$L_{DAE}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - f_{\theta}(g_{\phi}(\tilde{\mathbf{x}}^{(i)})))^2 \quad (2)$$

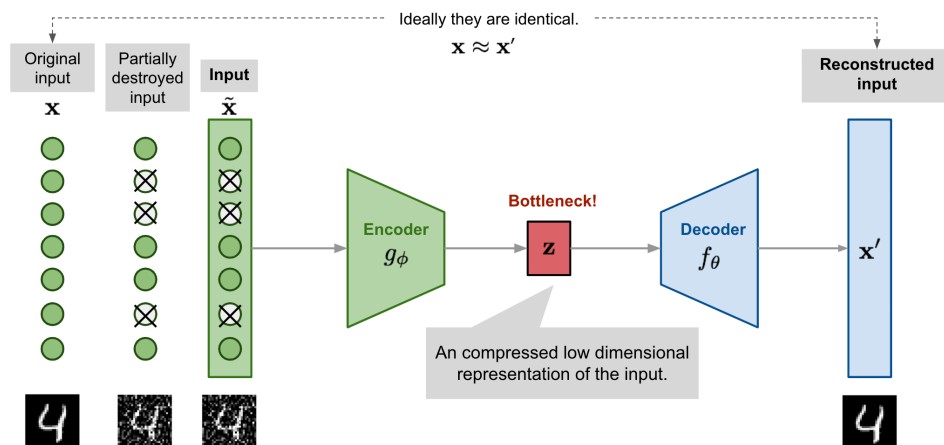


图 1-2 去噪自动编码器模型架构示意图

1.1.3 稀疏自动编码器 (Sparse Autoencoder, SAE)

稀疏自动编码器通过在损失函数中增加稀疏约束，强制隐层神经元在大部分时间处于“未激活”状态。假设隐藏层神经元 j 的平均激活度为 $\hat{\rho}_j$ ，我们希望它接近一个很小的稀疏参数 ρ （如 0.05）。通常使用 KL 散度作为惩罚项：

$$L_{\text{SAE}}(\theta) = L(\theta) + \beta \sum_{j=1}^{s_l} D_{\text{KL}}(\rho \parallel \hat{\rho}_j) \quad (3)$$

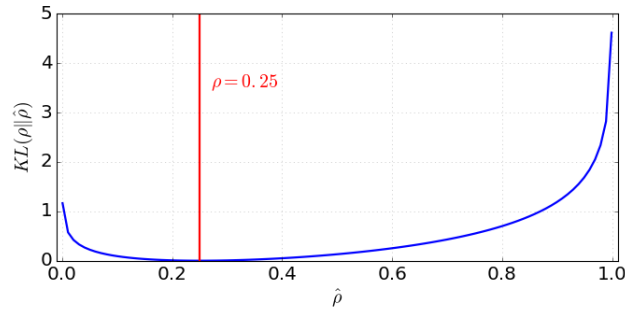


图 1-3 伯努利分布均值为 $\rho = 0.25$ 和均值为 $0 \leq \hat{\rho} \leq 1$ 的伯努利分布之间的 KL 散度

此外，还有 k -稀疏自动编码器 (k -Sparse Autoencoder)。其前向传播步骤如下：

1. 运行编码器得到潜在代码 $\mathbf{z} = g(\mathbf{x})$ 。
2. 对 $\mathbf{z}$ 进行排序，仅保留前 k 个最大的激活值，将其余值置零，得到 $\mathbf{z}' = \text{Sparsify}(\mathbf{z})$ 。
3. 计算重构损失 $L = \|\mathbf{x} - f(\mathbf{z}')\|_2^2$ 。

注意：反向传播时，梯度仅通过这前 k 个激活的神经元传递。

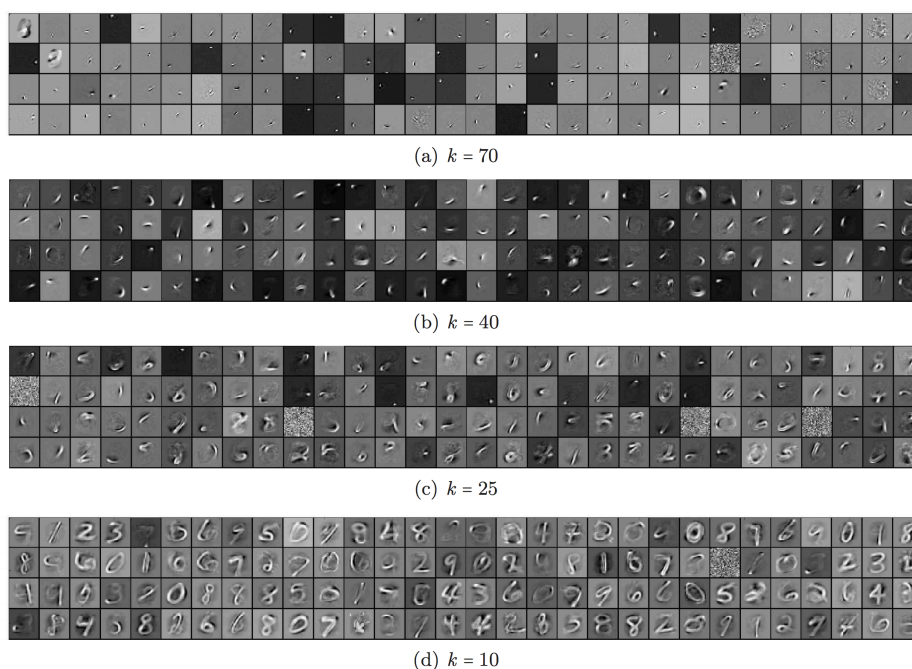


图 1-4 对不同稀疏级别 k 的 MNIST 数据集学习的 k -稀疏自动编码器的滤波器

1.1.4 收缩自动编码器 (Contracting Autoencoder, CAE)

收缩自动编码器旨在使学习到的表示对输入的微小扰动具有鲁棒性。它在损失函数中加入雅可比矩阵 (Jacobian Matrix) 的 Frobenius 范数作为惩罚项, 抑制编码器激活值对输入的敏感度:

$$\|J_f(\mathbf{x})\|_F^2 = \sum_{ij} \left(\frac{\partial h_j(\mathbf{x})}{\partial x_i} \right)^2 \quad (4)$$

这促使模型学习到的特征分布在低维流形上, 而对流形正交方向的变化保持不变。

1.2 VAE 原理详解 (The Principle of VAE)

Definition 1.2. 变分自编码器 (Variational Autoencoder, VAE) 是一种强大的深度生成模型, 它巧妙地将深度学习的表示能力与贝叶斯推断的概率框架相结合。与传统的自编码器 (Autoencoder) 不同, VAE 并非学习一个从输入到编码的确定性映射, 而是学习一个从输入到隐变量后验概率分布的映射。通过从这个学习到的分布中采样, VAE 能够生成与训练数据相似但全新的数据样本。

关键词: 生成模型, 变分自编码器, 贝叶斯推断, 证据下界, 重参数化技巧

1.2.1 引言 (Introduction)

生成模型的根本目标是学习观测数据 X 的真实概率分布 $P_{data}(X)$ 。一旦这个分布被成功建模, 我们便可以从中采样, 生成新的数据。早期的生成模型如受限玻尔兹曼机 挑战

(RBM) 面临训练困难的问题, 而传统的自编码器 (AE) 虽能学习数据的有效压缩表示, 但其隐空间 (latent space) 缺乏良好的结构性, 无法直接用于生成新样本。

Note 1.2 (VAE 的核心思想). 为了解决这一问题, Kingma 和 Welling 在 2013 年提出了变分自编码器。VAE 的核心思想是, 不再将输入数据 X 编码为一个确定的隐向量 z , 而是将其编码为一个概率分布 (通常是高斯分布)。具体来说, 模型学习这个分布的参数 (例如均值 μ 和方差 σ^2)。生成过程则变为:

1. 首先从这个分布中采样一个隐向量 z 。
2. 然后解码器 (Decoder) 基于这个 z 来重构出数据 $\hat{X}$ 。

通过这种方式, VAE 不仅学习了如何重构数据, 更重要的是, 它学习到了一个结构化、连续的隐空间, 使得我们可以在这个空间中进行采样和插值, 从而生成多样化的新数据。

1.2.2 模型架构与概率框架 (Model Architecture & Probabilistic Framework)

从概率的视角来看, VAE 假设所有的数据样本 X 是由一个我们无法直接观测到的隐变量 z 生成的。这个生成过程包含以下步骤:

1. 从一个简单的先验分布 $p(z)$ 中采样一个隐变量 z 。通常, 我们选择标准正态分布, 即 $z \sim \mathcal{N}(0, I)$ 。
2. 根据隐变量 z , 通过一个以神经网络 (解码器) 参数化的条件概率分布 $p_\theta(X|z)$ 来生成数据 X 。

我们的最终目标是最大化观测数据的边际似然 $p(X)$, 即所有可能隐变量 z 生成 X 的概率的积分:

$$p(X) = \int p_\theta(X|z)p(z)dz \quad (5)$$

然而, 这个积分通常是难以计算的 (intractable), 因为它需要在整个高维的隐空间上进行。这就引出了变分推断 (Variational Inference) 的必要性。

VAE 模型架构 VAE 由两个核心部分组成, 通常由深度神经网络实现:

- **编码器 (Encoder), 或称推断网络 (Inference Network):**
 - 输入: 数据样本 X 。
 - 输出: 隐变量 z 的后验分布 $p(z|X)$ 的近似分布 $q_\phi(z|X)$ 的参数。这里, ϕ 是编码器网络的参数。具体地, 编码器输出近似后验分布的参数, 例如高斯分布的均值 μ_X 和对数方差 $\log(\sigma_X^2)$ 。即 $q_\phi(z|X) = \mathcal{N}(z; \mu_X, \sigma_X^2 I)$ 。
- **解码器 (Decoder), 或称生成网络 (Generative Network):**

- 输入：从分布 $q_\phi(z|X)$ 中采样的隐向量 z 。
- 输出：重构的数据 $\hat{X}$ ，或者更准确地说，是条件概率分布 $p_\theta(X|z)$ 的参数。

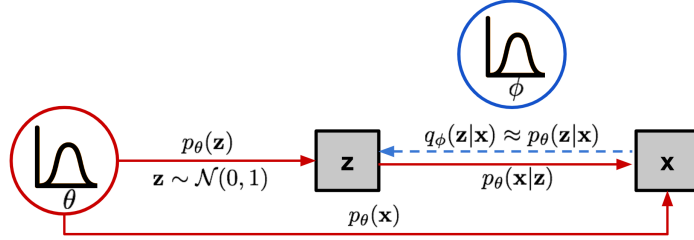


图 1-5 变分自动编码器中涉及的概率图模型

1.2.3 核心推导：证据下界 (Derivation of the Evidence Lower Bound, ELBO)

由于直接最大化 $p(X)$ 不可行，我们转而最大化它的一个下界，即**证据下界 (ELBO)**。我们从对数边际似然 $\log p(X)$ 开始，引入近似后验 $q_\phi(z|X)$ ，并根据詹森不等式 (Jensen's Inequality)，得到：

$$\begin{aligned} \log p(X) &\geq \int q_\phi(z|X) \log \frac{p_\theta(X|z)p(z)}{q_\phi(z|X)} dz \\ &= \mathbb{E}_{z \sim q_\phi(z|X)} \left[\log \frac{p_\theta(X|z)p(z)}{q_\phi(z|X)} \right] \end{aligned} \quad (6)$$

不等式的右侧就是证据下界 (ELBO)，记为 $\mathcal{L}(\phi, \theta; X)$ 。

Note 1.3 (ELBO 的分解).

$$\begin{aligned} \mathcal{L}(\phi, \theta; X) &= \mathbb{E}_{z \sim q_\phi(z|X)} [\log p_\theta(X|z) + \log p(z) - \log q_\phi(z|X)] \\ &= \underbrace{\mathbb{E}_{z \sim q_\phi(z|X)} [\log p_\theta(X|z)]}_{\text{重构项}} - \underbrace{D_{KL}(q_\phi(z|X) || p(z))}_{\text{KL 散度正则化项}} \end{aligned} \quad (7)$$

Note 1.4 (为什么使用反向 KL 散度？). 我们通过最小化 $D_{KL}(q_\phi(z|X) || p_\theta(z|X))$ 来逼近真实后验，这被称为**反向 KL 散度 (Reverse KL)**。

- **前向 KL ($D_{KL}(P||Q)$):** 要求近似分布 Q 覆盖真实分布 P 的所有非零区域。
- **反向 KL ($D_{KL}(Q||P)$):** 倾向于将近似分布 Q “挤压” 在真实分布 P 的峰值下方。这使得 q_ϕ 能够捕捉到 p_θ 的主要模式 (Mode-seeking behavior)，对于生成任务更为稳健。

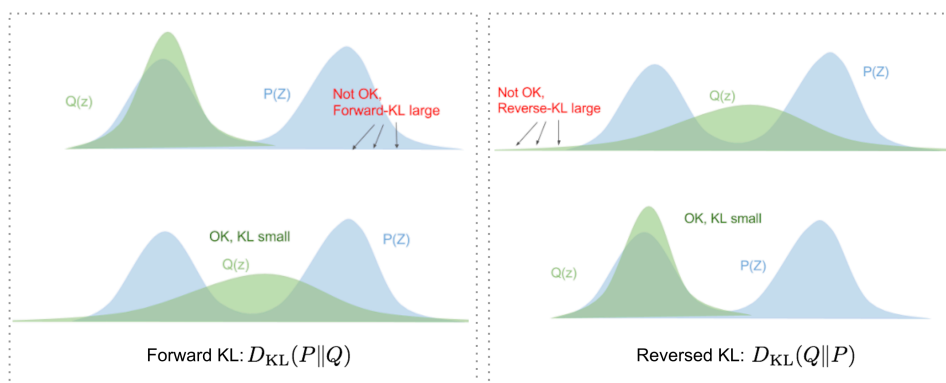


图 1-6 前向和反向 KL 散度对匹配两个分布有不同的要求

Note 1.5 (目标函数解读). VAE 的训练目标是最大化 ELBO，它由两部分构成：

- **重构项 (Reconstruction Term):** 该项最大化在给定隐变量 z 的情况下，重构出原始数据 X 的对数似然。这驱使解码器学习如何精确地从 z 恢复 X 。在实践中，常使用均方误差 (MSE) 或二元交叉熵 (BCE) 损失的负值。
- **KL 散度正则化项 (KL Divergence Regularization Term):** 该项衡量近似后验分布 $q_\phi(z|X)$ 与先验分布 $p(z)$ (通常是 $\mathcal{N}(0, I)$) 之间的差异。最小化 KL 散度，相当于对编码器施加约束，使其产生的隐变量分布接近标准正态分布，从而使隐空间变得平滑和连续。

因此，训练目标可以概括为：最大化重构质量的同时，保持隐空间分布的规整性。

1.2.4 关键技术：重参数化技巧 (The Reparameterization Trick)

Definition 1.3 (重参数化技巧). 在目标函数中，从 $q_\phi(z|X)$ 中采样 z 的过程是随机的，不可微分，这会阻碍梯度的反向传播。**重参数化技巧**通过将随机性与模型参数分离来解决此问题。对于高斯分布 $z \sim \mathcal{N}(z; \mu_X, \sigma_X^2 I)$ ，其采样过程可重写为：

1. 从一个固定的标准正态分布中采样噪声 $\epsilon \sim \mathcal{N}(0, I)$ 。
2. 通过确定性变换生成 z : $z = \mu_X + \sigma_X \odot \epsilon$ 。

这样，随机采样被移出计算图，梯度可以顺利地损失函数回传到编码器的参数 ϕ (即 μ_X 和 σ_X)。

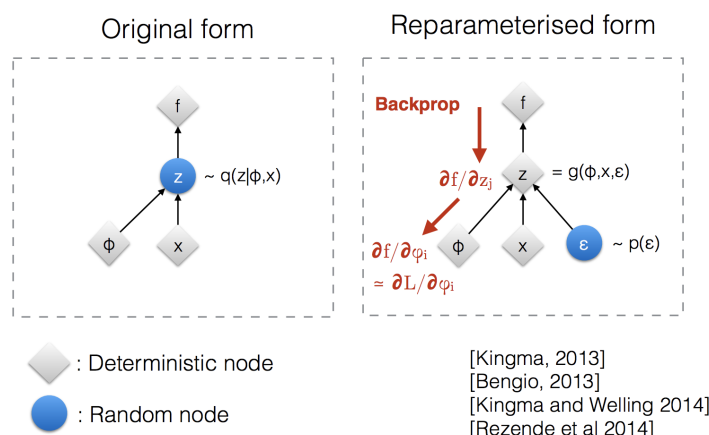


图 1-7 重参数化技巧示意图

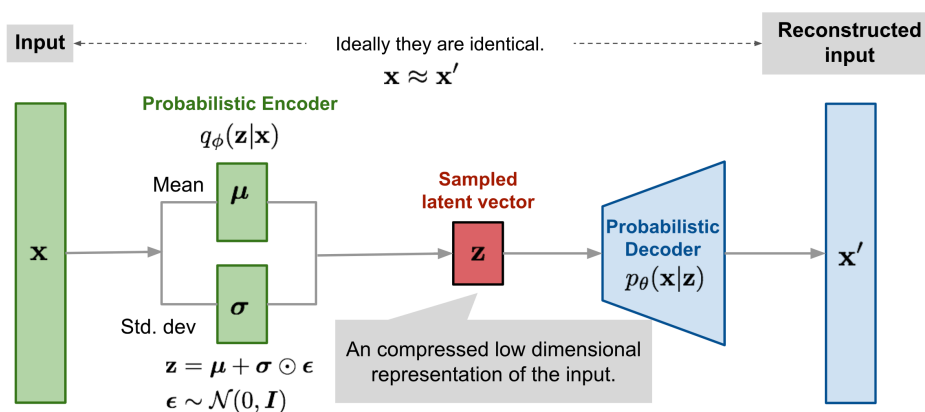


图 1-8 假设多元高斯分布的变分自动编码器模型示意图

1.2.5 结论与讨论 (Conclusion and Discussion)

变分自编码器（VAE）通过最大化证据下界（ELBO）成功地将深度神经网络与概率图模型结合起来。其损失函数由重构损失和 KL 散度正则化项构成，分别保证了模型的重构能力和隐空间的良好结构。关键的重参数化技巧使得模型能够通过标准的梯度下降算法进行端到端的训练。

Note 1.6 (VAE vs. GAN). 与 GAN 相比，VAE 的训练过程更稳定，能够显式地学习数据的概率密度。然而，VAE 生成的样本通常比 GAN 生成的样本模糊，这是因为其损失函数倾向于覆盖数据的所有模式，导致在细节上有所妥协。

尽管如此，VAE 作为一种 foundational 的深度生成模型，其思想，特别是对隐空间的概率建模，对后续的生成模型（包括扩散模型）产生了深远的影响。理解 VAE 是掌握现代生成式 AI 版图不可或缺的一步。

1.3 进阶变分模型 (Advanced VAE Models)

1.3.1 Beta-VAE (β -VAE)

Definition 1.4 (解耦表示 (Disentangled Representation)). 如果隐变量 $\mathbf{z}$ 中的每个维度仅对数据的一个生成因子敏感 (例如一个维度控制“发色”，另一个控制“肤色”)，且对其他因子保持不变，则称这种表示为解耦的。解耦表示具有极佳的可解释性。

β -VAE 旨在通过调整 ELBO 中的 KL 散度项权重来发现解耦的潜在因子。无论是从直觉上增加 KL 惩罚，还是从约束优化的角度来看，都有坚实的理论基础。我们希望最大化生成概率，同时约束后验分布与先验分布的距离小于 δ ：

$$\max_{\phi, \theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z})] \quad \text{subject to } D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z})) < \delta \quad (8)$$

使用拉格朗日乘数法，这等价于最大化：

$$\mathcal{F}(\theta, \phi, \beta) = \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) - \beta(D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z})) - \delta) \quad (9)$$

去掉常数项，得到 β -VAE 的损失函数：

$$L_{\beta\text{-VAE}} = -\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})] + \beta D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z})) \quad (10)$$

Note 1.7 (β 的作用). • 当 $\beta = 1$ 时，模型退化为标准的 VAE。

- 当 $\beta > 1$ 时，模型对隐变量施加了更强的独立性约束，鼓励学习更有效的、解耦的潜在编码。但这通常会带来重构质量与解耦程度之间的权衡 (Trade-off)。

1.3.2 VQ-VAE (Vector Quantized VAE)

传统的 VAE 使用连续的隐变量分布 (如高斯分布)，这可能不适合某些模态 (如语言、推理)。VQ-VAE 引入了离散潜在变量。

Note 1.8 (向量量化 (Vector Quantization)). VQ-VAE 维护一个“代码簿” (Codebook)，包含 K 个嵌入向量 $\mathbf{e}_i$ 。编码器的输出 $\mathbf{z}_e(\mathbf{x})$ 不再直接传给解码器，而是通过最近邻搜索映射到代码簿中最接近的嵌入向量 $\mathbf{e}_k$ ：

$$\mathbf{z}_q(\mathbf{x}) = \mathbf{e}_k, \quad \text{where } k = \arg \min_i \|\mathbf{z}_e(\mathbf{x}) - \mathbf{e}_i\|_2 \quad (11)$$

由于 $\arg \min$ 操作不可导，VQ-VAE 使用直通估计器 (Straight-Through Estimator) 将解码器的梯度直接复制回编码器 (即 $\nabla_{\mathbf{z}} L \approx \nabla_{\mathbf{z}_q} L$)。其总损失函数如下 (其中 $\text{sg}[\cdot]$ 表示这个梯度的截断算子 stop gradient)：

$$L = \underbrace{\|\mathbf{x} - D(\mathbf{e}_k)\|_2^2}_{\text{Reconstruction Loss}} + \underbrace{\|\text{sg}[E(\mathbf{x})] - \mathbf{e}_k\|_2^2}_{\text{VQ Loss}} + \underbrace{\beta \|E(\mathbf{x}) - \text{sg}[\mathbf{e}_k]\|_2^2}_{\text{Commitment Loss}} \quad (12)$$

代码簿更新 (EMA): 为了更稳定地更新嵌入向量, 可以使用指数移动平均 (Exponential Moving Average):

$$N_i^{(t)} = \gamma N_i^{(t-1)} + (1 - \gamma) n_i^{(t)}, \quad \mathbf{m}_i^{(t)} = \gamma \mathbf{m}_i^{(t-1)} + (1 - \gamma) \sum_j \mathbf{z}_{i,j}^{(t)}, \quad \mathbf{e}_i^{(t)} = \frac{\mathbf{m}_i^{(t)}}{N_i^{(t)}} \quad (13)$$

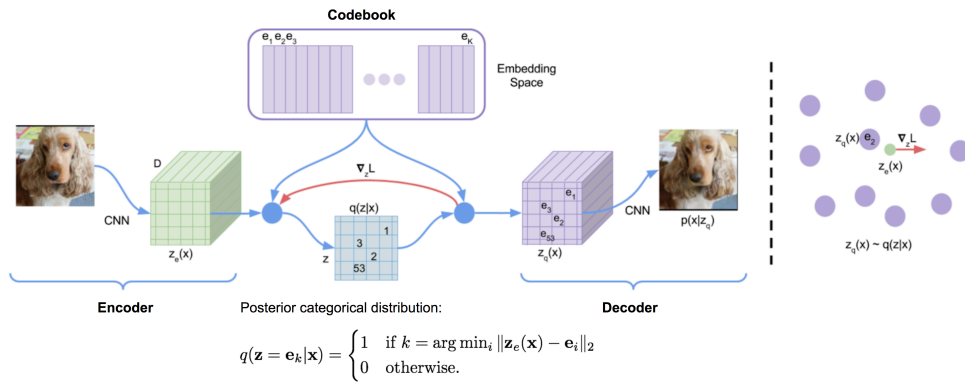


图 1-9 VQ-VAE 的架构

VQ-VAE-2 进一步引入了层级结构 (Hierarchical VQ-VAE), 分离局部纹理和全局形状信息, 并结合自回归模型 (如 PixelCNN) 来学习离散编码的先验分布, 生成了极高质量的图像。

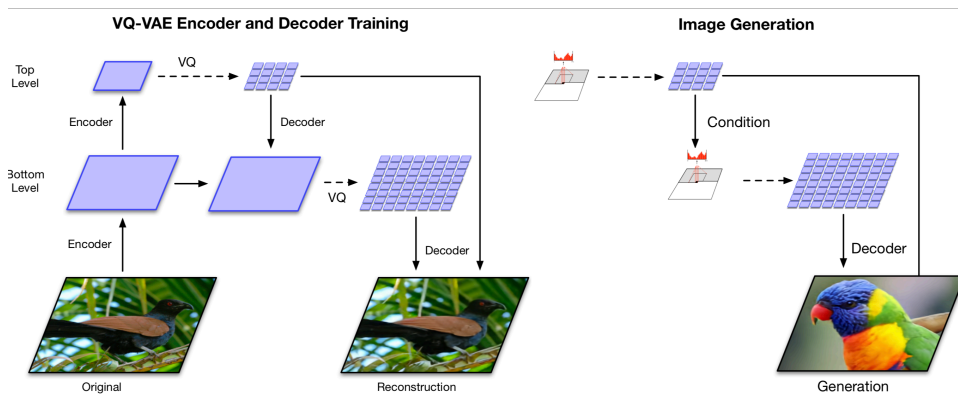


图 1-10 层次 VQ-VAE 和多阶段图像生成的架构

1.3.3 TD-VAE (Temporal Difference VAE)

TD-VAE 专为序列数据设计, 旨在解决传统序列模型推断效率低的问题。它基于三个核心思想:

1. **状态空间模型 (State Space Model):** 假设观测序列由一系列不可见的隐状态控制。
2. **信念状态 (Belief State):** 智能体应学会将过去的所有信息编码为一个信念状态 b_t , 用于推理未来。

3. **跳跃预测 (Jumpy Prediction):** 模型应具备根据当前信息直接预测遥远未来的能力，而无需一步步模拟中间过程。

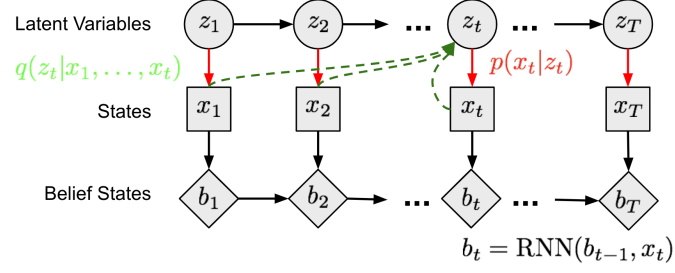


图 1-11 作为马尔可夫链模型的状态空间模型

TD-VAE 需要学习四种分布：

- **解码器分布** $p_D(x_t|z_t)$: 从隐状态生成观测。
- **转移分布** $p_T(z_t|z_{t-1})$: 捕捉隐变量的序列依赖。
- **信念分布** $p_B(z_t|b_t)$: 从信念状态推断隐变量。
- **平滑分布** $p_S(z_{t-1}|z_t, b_{t-1}, b_t)$: 回溯推断。

TD-VAE 的最终目标函数是在两个时间戳 $t_1 < t_2$ 之间最大化以下期望 (Jump Prediction)：

$$J_{t_1, t_2} = \mathbb{E}[\log p_D(x_{t_2}|z_{t_2}) + \log p_B(z_{t_1}|b_{t_1}) + \log p_T(z_{t_2}|z_{t_1}) - \log p_B(z_{t_2}|b_{t_2}) - \log p_S(z_{t_1}|z_{t_2}, b_{t_1}, b_{t_2})] \quad (14)$$

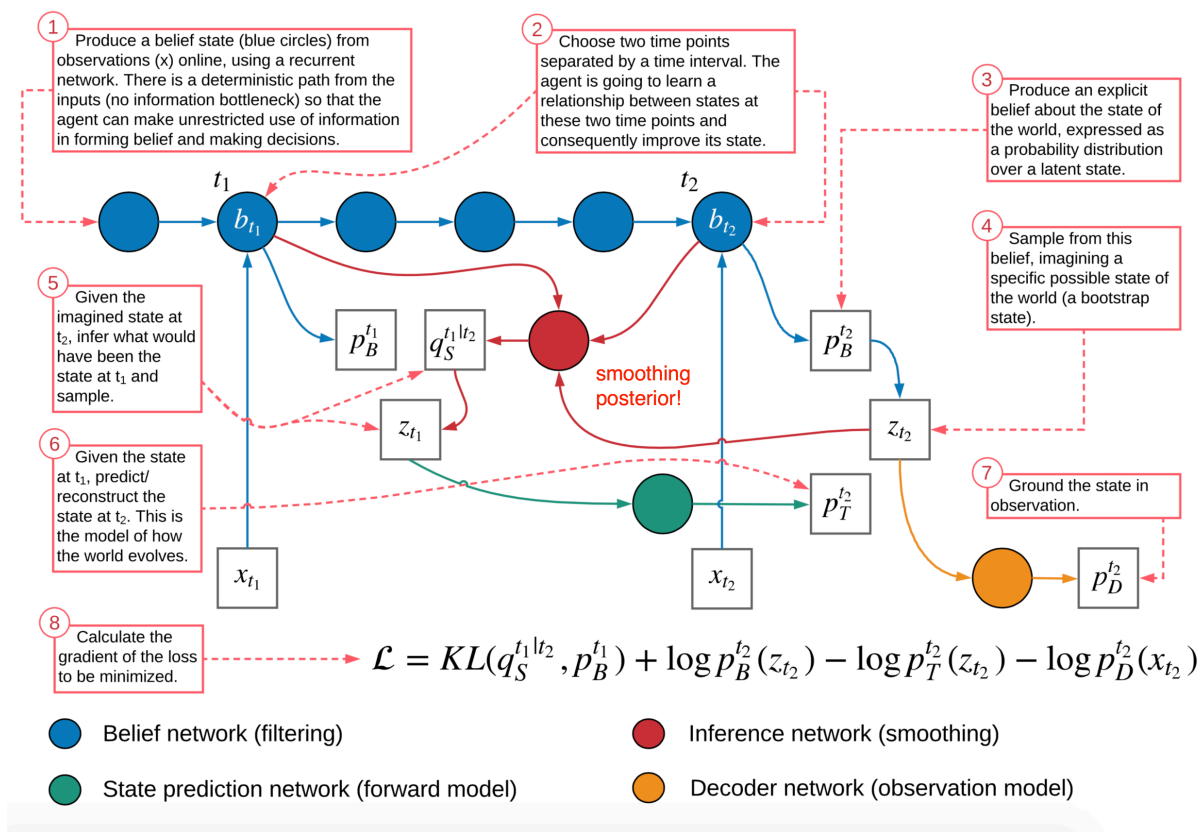


图 1-12 TD-VAE 架构详细概览

2 扩散模型 (Diffusion Models)

表 2-1 符号说明

符号	含义
$\mathbf{x}_0$	真实数据样本
$\mathbf{x}_t$	时间步 t 的噪声样本
T	总扩散步数
β_t	方差调度表, 控制加噪量
α_t	$1 - \beta_t$
$\bar{\alpha}_t$	$\prod_{i=1}^t \alpha_i$, 累积乘积
ϵ	标准高斯噪声, $\mathcal{N}(\mathbf{0}, \mathbf{I})$
ϵ_θ	噪声预测网络 (或分数网络)
$q(\mathbf{x}_t \mathbf{x}_{t-1})$	前向扩散过程 (加噪)
$p_\theta(\mathbf{x}_{t-1} \mathbf{x}_t)$	逆向扩散过程 (去噪)
L_{simple}	简化的均方误差损失

迄今为止，必须要提到的生成模型有三类：GAN、VAE 和基于流的模型（Flow-based models）。它们在生成高质量样本方面取得了巨大成功，但各有限制。GAN 因其对抗性训练性质，训练可能不稳定且生成多样性较差；VAE 依赖于代理损失；流模型则必须使用专门的架构来构建可逆变换。

扩散模型受非平衡热力学启发。它们定义了一个扩散步骤的马尔可夫链，通过缓慢地向数据添加随机噪声，然后学习逆向扩散过程，从噪声中构建所需的数据样本。与 VAE 或流模型不同，扩散模型是通过固定的过程学习的，并且隐变量具有高维度（与原始数据相同）。

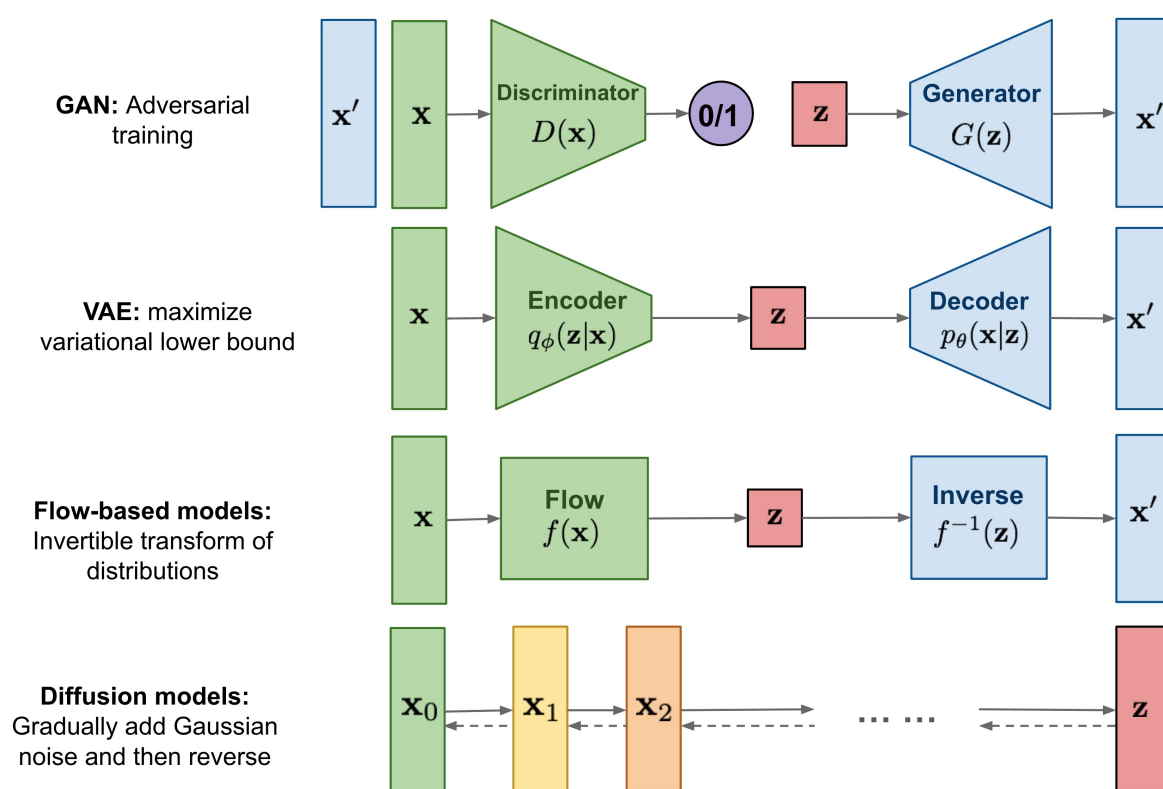


图 2-1 不同类型生成模型的概览：GAN、VAE、Flow-based 和 Diffusion

2.1 基本原理

包括扩散概率模型 (Sohl-Dickstein et al., 2015)、噪声条件分数网络 (NCSN; Yang & Ermon, 2019) 和去噪扩散概率模型 (DDPM; Ho et al. 2020) 在内的多种基于扩散的生成模型，其底层思想是相似的。

2.1.1 前向扩散过程 (Forward Diffusion Process)

给定从真实数据分布中采样的数据点 $\mathbf{x}_0 \sim q(\mathbf{x})$ ，我们定义一个前向扩散过程，在 T 个步骤中向样本添加少量高斯噪声，产生一系列噪声样本 $\mathbf{x}_1, \dots, \mathbf{x}_T$ 。步长由方差调度 $\{\beta_t \in (0, 1)\}$ 控制：

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}) \quad (15)$$

其中：

- $\mathbf{x}_t$ 表示在时间步 t 加入噪声后的图像数据。
- β_t 是控制方差的超参数，决定了每一步加入多少噪声。
- $\mathcal{N}(\cdot; \mu, \Sigma)$ 表示高斯（正态）分布，均值为 μ ，协方差矩阵为 Σ 。

实际上，上述公式可以写成重参数化的形式：

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_{t-1}, \quad \text{其中 } \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (16)$$

Note 2.1 (关于系数根号的解释). 为什么系数是 $\sqrt{1-\beta_t}$ 而不是 $1-\beta_t$? 这是为了保持方差稳定。假设输入 $\mathbf{x}_{t-1}$ 已经过标准化，即方差为 1。如果我们希望加噪后的 $\mathbf{x}_t$ 方差仍然保持为 1，根据方差性质 $\text{Var}(aX + bY) = a^2\text{Var}(X) + b^2\text{Var}(Y)$ （假设独立）：

$$\text{Var}(\mathbf{x}_t) = (\underbrace{(\sqrt{1-\beta_t})^2}_{\approx 1} \cdot \underbrace{\text{Var}(\mathbf{x}_{t-1})}_{\approx 1}) + (\underbrace{(\sqrt{\beta_t})^2}_{1} \cdot \underbrace{\text{Var}(\epsilon)}_{1}) = (1-\beta_t) + \beta_t = 1 \quad (17)$$

如果不加根号，随着扩散步骤 t 的增加，信号的方差会迅速衰减或爆炸。

随着步长 t 变大，数据样本 $\mathbf{x}_0$ 逐渐失去其可辨别的特征。最终当 $T \rightarrow \infty$ 时， $\mathbf{x}_T$ 等价于各向同性高斯分布。

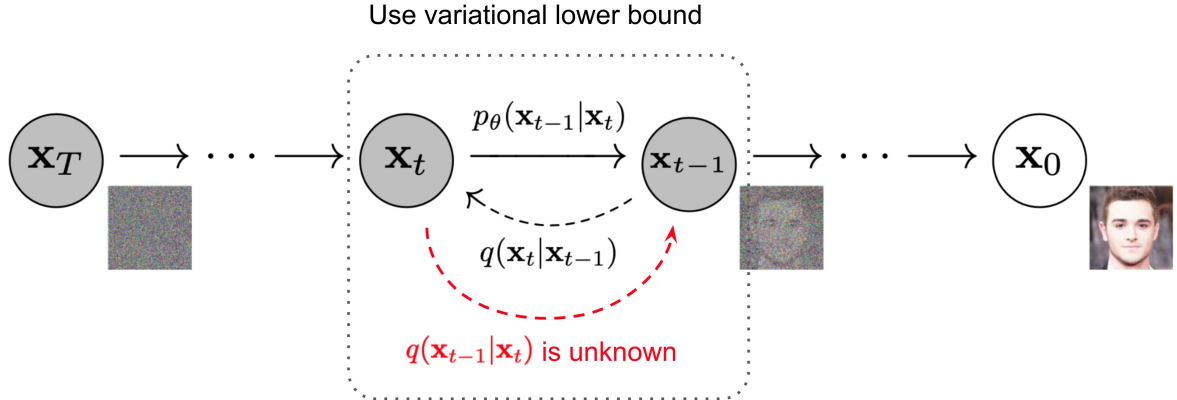


图 2-2 前向（逆向）扩散过程的马尔可夫链，通过缓慢添加（去除）噪声来生成样本

上述过程的一个优良特性是，我们可以使用重参数化技巧以闭式解 (**Closed-form**) 在任意时间步 t 对 $\mathbf{x}_t$ 进行采样。这意味着我们不需要迭代 t 次（即 $\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{x}_t$ ），而是可以直接从 $\mathbf{x}_0$ 一步计算出 $\mathbf{x}_t$ 。

令 $\alpha_t = 1 - \beta_t$ 且 $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ 。推导如下：

首先，根据前向过程定义：

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (18)$$

如果我们把 $\mathbf{x}_{t-1}$ 也展开（即代入 $\mathbf{x}_{t-1} = \sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}$ ）：

$$\begin{aligned}\mathbf{x}_t &= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\ &= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \underbrace{\sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1}}_{\text{两个高斯分布的混合}}\end{aligned}\quad (19)$$

根据高斯分布的可加性：若 $X \sim \mathcal{N}(0, \sigma_1^2 \mathbf{I})$, $Y \sim \mathcal{N}(0, \sigma_2^2 \mathbf{I})$ 且相互独立，则 $X + Y \sim \mathcal{N}(0, (\sigma_1^2 + \sigma_2^2) \mathbf{I})$ 。上式中，由于 ϵ 服从标准正态分布，我们可以合并噪声项：

$$\begin{aligned}\text{方差} &= (\sqrt{\alpha_t(1 - \alpha_{t-1})})^2 + (\sqrt{1 - \alpha_t})^2 \\ &= \alpha_t(1 - \alpha_{t-1}) + 1 - \alpha_t \\ &= \alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t \\ &= 1 - \alpha_t\alpha_{t-1}\end{aligned}\quad (20)$$

因此，这一步合并后的噪声项可以表示为 $\sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2}$ （其中 $\bar{\epsilon}_{t-2} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ）。

如果我们不断递归展开直到 $t = 0$ ，所有的系数连乘起来就是累积乘积 $\bar{\alpha}_t$ ：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad (21)$$

这对应着边缘分布：

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (22)$$

以下代码实现了 β_t 调度器的定义、参数的预计算以及前向扩散采样 `q_sample`：

```
1 import torch
2 import torch.nn.functional as F
3
4 def cosine_beta_schedule(timesteps, s=0.008):
5     """
6     cosine schedule as proposed in https://arxiv.org/abs/2102.09672
7     """
8     steps = timesteps + 1
9     x = torch.linspace(0, timesteps, steps)
10    alphas_cumprod = torch.cos(((x / timesteps) + s) / (1 + s) * torch.pi * 0.5)
11    ↪ ** 2
12    alphas_cumprod = alphas_cumprod / alphas_cumprod[0]
13    betas = 1 - (alphas_cumprod[1:] / alphas_cumprod[:-1])
14    return torch.clip(betas, 0.0001, 0.9999)
15
16 def linear_beta_schedule(timesteps):
17     beta_start = 0.0001
18     beta_end = 0.02
19     return torch.linspace(beta_start, beta_end, timesteps)
20
21 timesteps = 200
```

```

22 # beta 调度器
23 betas = linear_beta_schedule(timesteps=timesteps)
24
25 # 计算 alphas
26 alphas = 1. - betas
27 alphas_cumprod = torch.cumprod(alphas, axis=0)
28 alphas_cumprod_prev = F.pad(alphas_cumprod[:-1], (1, 0), value=1.0)
29 sqrt_recip_alphas = torch.sqrt(1.0 / alphas)
30
31 # calculations for diffusion q(x_t | x_{t-1}) and others
32 sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
33 sqrt_one_minus_alphas_cumprod = torch.sqrt(1. - alphas_cumprod)
34
35 # calculations for posterior q(x_{t-1} | x_t, x_0)
36 posterior_variance = betas * (1. - alphas_cumprod_prev) / (1. - alphas_cumprod)
37
38 def extract(a, t, x_shape):
39     batch_size = t.shape[0]
40     out = a.gather(-1, t.cpu())
41     return out.reshape(batch_size, *((1,) * (len(x_shape) - 1))).to(t.device)
42
43 # forward diffusion
44 def q_sample(x_start, t, noise=None):
45     if noise is None:
46         noise = torch.randn_like(x_start)
47
48     sqrt_alphas_cumprod_t = extract(sqrt_alphas_cumprod, t, x_start.shape)
49     sqrt_one_minus_alphas_cumprod_t = extract(
50         sqrt_one_minus_alphas_cumprod, t, x_start.shape
51     )
52
53     return sqrt_alphas_cumprod_t * x_start + sqrt_one_minus_alphas_cumprod_t *
    ↪ noise

```

(*) 提示: 当我们合并两个具有不同方差的高斯分布 $\mathcal{N}(\mathbf{0}, \sigma_1^2 \mathbf{I})$ 和 $\mathcal{N}(\mathbf{0}, \sigma_2^2 \mathbf{I})$ 时, 新的分布是 $\mathcal{N}(\mathbf{0}, (\sigma_1^2 + \sigma_2^2) \mathbf{I})$ 。在这里合并的标准差是 $\sqrt{(1 - \alpha_t) + \alpha_t(1 - \alpha_{t-1})} = \sqrt{1 - \alpha_t \alpha_{t-1}}$ 。通常, 当样本变得更嘈杂时, 我们可以承受更大的更新步长, 所以 $\beta_1 < \beta_2 < \dots < \beta_T$, 因此 $\bar{\alpha}_1 > \dots > \bar{\alpha}_T$ 。

2.1.2 逆向扩散过程 (Reverse Diffusion Process)

如果我们能够逆转上述过程并从 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 中采样, 我们就能够从高斯噪声输入 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 一步步去噪, 最终重建出服从真实数据分布的样本。

Note 2.2 (为什么逆向过程难以直接计算?)。根据贝叶斯公式, 真实的逆向后验分布为:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)} \quad (23)$$

虽然 $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ 是已知的 (前向加噪过程), 但我们要计算这个后验分布, 需要知道边缘分布 $q(\mathbf{x}_t)$ 和 $q(\mathbf{x}_{t-1})$ 。为了得到这些边缘分布, 我们需要对整个数据分布 $q(\mathbf{x}_0)$ 进行

积分：

$$q(\mathbf{x}_t) = \int q(\mathbf{x}_t|\mathbf{x}_0)q(\mathbf{x}_0)d\mathbf{x}_0 \quad (24)$$

这相当于需要遍历整个数据集（甚至包含未观测到的真实数据），因此直接计算 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 是 **不可行（Intractable）** 的。

虽然我们无法直接计算真实的 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ ，但如果 β_t 足够小，这个分布近似于高斯分布。因此，我们可以训练一个深度学习模型 p_θ 来近似这个条件概率分布。模型输入当前的含噪图像 $\mathbf{x}_t$ 和时间步 t ，输出上一时刻图像 $\mathbf{x}_{t-1}$ 的分布参数（均值和方差）。

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \quad (25)$$

其中：

- $p_\theta(\mathbf{x}_{0:T})$ 是逆向过程的联合概率分布，表示从纯噪声 $\mathbf{x}_T$ 到生成数据 $\mathbf{x}_0$ 的完整轨迹。
- $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ 是逆向过程的起点，即我们要采样的标准高斯噪声。
- $\prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 体现了马尔可夫性质，即当前时刻的状态只依赖于上一时刻（逆向来看是 t 到 $t-1$ ）。

关键在于每个逆向步骤 $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 也被建模为高斯分布（因为当 β_t 很小时，逆向过程近似高斯）：

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)) \quad (26)$$

这里有两个可学习的参数（由神经网络拟合）：

1. **均值 $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$** ：这是模型的核心预测目标。我们需要训练一个神经网络（通常是 U-Net），输入当前的噪声图 $\mathbf{x}_t$ 和时间步 t ，预测出这一步去噪后的均值应该在哪里。
2. **方差 $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$** ：在 DDPM 的原始论文中，作者发现将方差固定为常数（如 $\beta_t \mathbf{I}$ 或 $\tilde{\beta}_t \mathbf{I}$ ）效果已经很好，因此通常不需要学习方差，或者仅学习一个插值系数。

通过从该参数化分布中采样，我们可以生成新的样本。这里的“重建真实样本”并非指恢复训练集中某张特定的图片，而是指生成的图片在统计规律上服从真实数据分布 $q(\mathbf{x}_0)$ ，即看起来像真实的图片。

Note 2.3 (神经网络如何“拟合”概率分布？). 这是一个常见的误解。神经网络 f_θ 本身是一个确定性的函数映射（给定相同的输入，永远输出相同的结果），它无法直接输出一个“随机分布”。

在扩散模型（以及 VAE）中，我们采用的是**参数化概率分布 (Parameterized Probability Distribution)** 的思想：

1. **预设分布形式**：我们假设目标分布服从某种特定形式（如高斯分布 $\mathcal{N}(\mu, \Sigma)$ ）。
2. **网络预测参数**：神经网络的任务不是生成随机数，而是预测该分布的**参数**。例如，输入 $\mathbf{x}_t$ ，网络输出均值向量 μ_θ 和对角方差 Σ_θ 。
3. **采样引入随机性**：一旦网络给出了确定性的参数 μ 和 Σ ，我们就可以使用重参数化技巧（或者直接从分布中采样）来引入随机性： $\mathbf{x}_{t-1} = \mu_\theta + \sqrt{\Sigma_\theta} \cdot \mathbf{z}$ ，其中 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 。

因此，当我们说“神经网络拟合分布”时，实际上是指用神经网络去拟合定义该分布的**参数**。

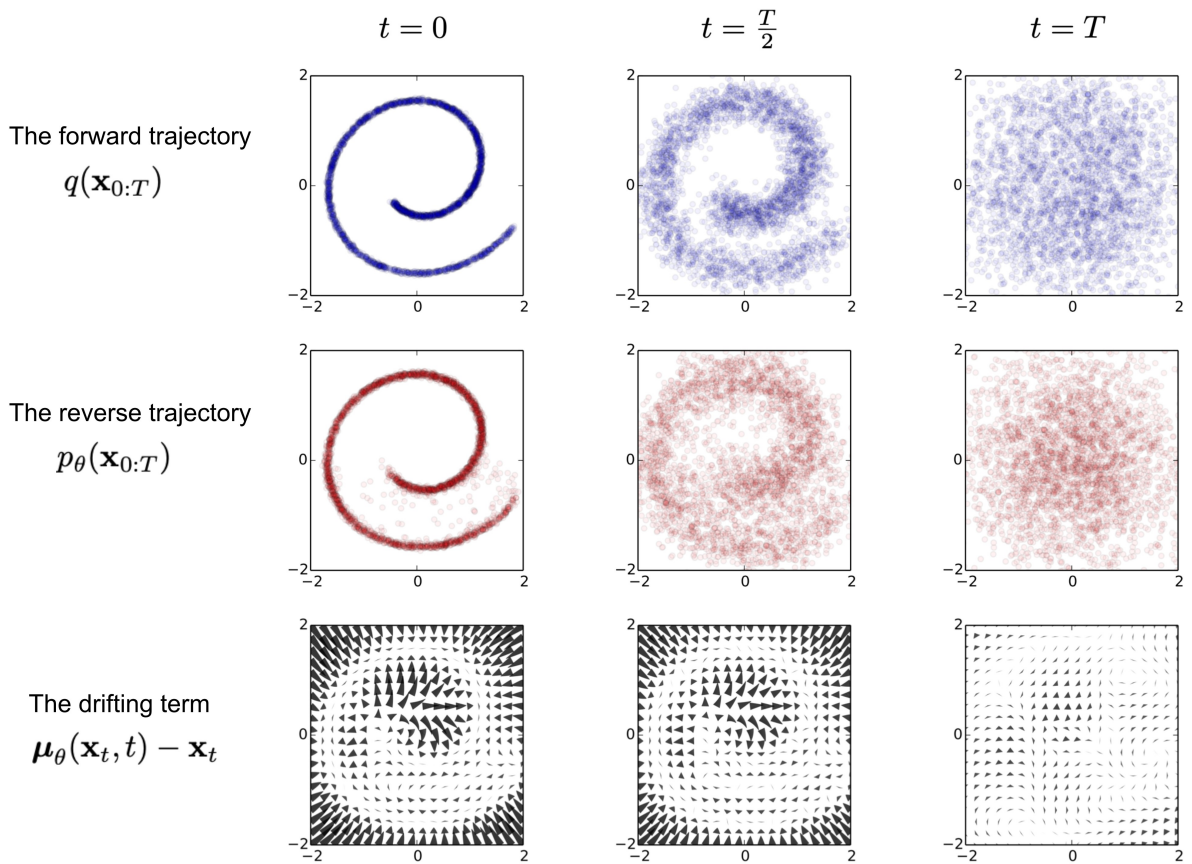


图 2-3 训练扩散模型以建模 2D 瑞士卷数据的示例

值得注意的是，当以 $\mathbf{x}_0$ 为条件时，逆向条件概率是可计算的：

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}) \quad (27)$$

使用贝叶斯规则：

$$\begin{aligned}
q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\
&\propto \exp\left(-\frac{1}{2}\left(\frac{(\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)^2}{1 - \bar{\alpha}_t}\right)\right) \\
&= \exp\left(-\frac{1}{2}\left(\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t}\mathbf{x}_t\mathbf{x}_{t-1} + \alpha_t\mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0\mathbf{x}_{t-1} + \bar{\alpha}_{t-1}\mathbf{x}_0^2}{1 - \bar{\alpha}_{t-1}} - C(\mathbf{x}_t, \mathbf{x}_0)\right)\right) \\
&= \exp\left(-\frac{1}{2}\left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right)\mathbf{x}_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}\mathbf{x}_0\right)\mathbf{x}_{t-1} + C'(\mathbf{x}_t, \mathbf{x}_0)\right)\right) \quad (28)
\end{aligned}$$

其中 $C(\mathbf{x}_t, \mathbf{x}_0)$ 和 $C'(\mathbf{x}_t, \mathbf{x}_0)$ 是不包含 $\mathbf{x}_{t-1}$ 的项。

根据标准高斯密度函数，我们可以得到均值和方差的参数化形式：

$$\tilde{\beta}_t = 1/\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right) = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t \quad (29)$$

以及：

$$\begin{aligned}
\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) &= \left(\frac{\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}\mathbf{x}_0\right) / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right) \\
&= \left(\frac{\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}\mathbf{x}_0\right) \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot \beta_t \\
&= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0
\end{aligned} \quad (30)$$

利用 $\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t)$ ，我们可以将均值表示为仅依赖于 $\mathbf{x}_t$ 和噪声 ϵ_t 的形式：

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_t\right) \quad (31)$$

以下是逆向过程的采样代码实现，包含 `p_sample`（单步采样）和 `p_sample_loop`（完整循环）：

```

1 @torch.no_grad()
2 def p_sample(model, x, t, t_index):
3     betas_t = extract(betas, t, x.shape)
4     sqrt_one_minus_alphas_cumprod_t = extract(
5         sqrt_one_minus_alphas_cumprod, t, x.shape
6     )
7     sqrt_recip_alphas_t = extract(sqrt_recip_alphas, t, x.shape)
8
9     # Equation 11 in the paper
10    # Use our model (noise predictor) to predict the mean
11    model_mean = sqrt_recip_alphas_t * (
12        x - betas_t * model(x, t) / sqrt_one_minus_alphas_cumprod_t
13    )
14

```



```

15     if t_index == 0:
16         return model_mean
17     else:
18         posterior_variance_t = extract(posterior_variance, t, x.shape)
19         noise = torch.randn_like(x)
20         # Algorithm 2 line 4:
21         return model_mean + torch.sqrt(posterior_variance_t) * noise
22
23 # Algorithm 2 but save all images:
24 @torch.no_grad()
25 def p_sample_loop(model, shape):
26     device = next(model.parameters()).device
27
28     b = shape[0]
29     # start from pure noise (for each example in the batch)
30     img = torch.randn(shape, device=device)
31     imgs = []
32
33     for i in tqdm(reversed(range(0, timesteps)), desc='sampling loop time step',
34                 ↪ total=timesteps):
35         img = p_sample(model, img, torch.full((b,), i, device=device,
36                 ↪ dtype=torch.long), i)
37         imgs.append(img.cpu().numpy())
38     return imgs
39
40 @torch.no_grad()
41 def sample(model, image_size, batch_size=16, channels=3):
42     return p_sample_loop(model, shape=(batch_size, channels, image_size,
43     ↪ image_size))

```

2.1.3 训练目标

这与 VAE 非常相似，因此我们可以使用变分下界（VLB）来优化负对数似然：

$$\begin{aligned}
 -\log p_{\theta}(\mathbf{x}_0) &\leq -\log p_{\theta}(\mathbf{x}_0) + D_{\text{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\
 &= -\log p_{\theta}(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{0:T})/p_{\theta}(\mathbf{x}_0)} \right] \\
 &= -\log p_{\theta}(\mathbf{x}_0) + \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{0:T})} + \log p_{\theta}(\mathbf{x}_0) \right] \\
 &= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_{\theta}(\mathbf{x}_{0:T})} \right] = L_{\text{VLB}}
 \end{aligned} \tag{32}$$

可以将 L_{VLB} 进一步重写为多个 KL 散度项的组合：

$$\begin{aligned}
L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \\
&= \mathbb{E}_q \left[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \left(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \cdot \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} \right) + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right] \\
&= \mathbb{E}_q \left[\underbrace{\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)}}_{L_T} + \underbrace{\sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}}_{L_{T-1}+\dots+L_1} + \underbrace{\log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}}_{L_0} \right] \tag{33}
\end{aligned}$$

每一项标记如下：

$$L_{\text{VLB}} = L_T + L_{T-1} + \dots + L_0 \tag{34}$$

其中：

- L_T 是常数（因为 q 无参数且 $\mathbf{x}_T$ 是高斯噪声），训练时可忽略。
- L_t 是两个高斯分布之间的 KL 散度，可有闭式解。
- L_0 通常使用独立的离散解码器来建模。

2.1.4 训练损失 L_t 的参数化

我们需要学习一个神经网络来近似逆向扩散过程中的条件概率分布 $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$ 。我们希望训练 $\boldsymbol{\mu}_\theta$ 来预测 $\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}}\boldsymbol{\epsilon}_t)$ 。因为 $\mathbf{x}_t$ 在训练时是作为输入可用的，我们可以改为重参数化高斯噪声项，使其根据输入 $\mathbf{x}_t$ 在时间步 t 预测 $\boldsymbol{\epsilon}_t$ ：

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) \quad (35)$$

因此 $\mathbf{x}_{t-1}$ 的采样过程变为:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \Sigma_{\theta}(\mathbf{x}_t, t)^{1/2} \mathbf{z} \quad (36)$$

损失项 L_t 被参数化为最小化与 $\tilde{\mu}_t$ 的差异:

$$\begin{aligned} L_t &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2 \|\Sigma_{\theta}(\mathbf{x}_t, t)\|_2^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_{\theta}(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2 \|\Sigma_{\theta}\|_2^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) - \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) \right\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{(1 - \alpha_t)^2}{2 \alpha_t (1 - \bar{\alpha}_t) \|\Sigma_{\theta}\|_2^2} \|\epsilon_t - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{(1 - \alpha_t)^2}{2 \alpha_t (1 - \bar{\alpha}_t) \|\Sigma_{\theta}\|_2^2} \left\| \epsilon_t - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t) \right\|^2 \right] \end{aligned} \quad (37)$$

2.1.5 简化 (Simplification)

Ho et al. (2020) 发现, 使用忽略加权项的简化目标进行扩散模型训练效果更好:

$$\begin{aligned} L_t^{\text{simple}} &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} [\|\epsilon_t - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2] \\ &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\left\| \epsilon_t - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t) \right\|^2 \right] \end{aligned} \quad (38)$$

最终的简单目标函数是:

$$L_{\text{simple}} = L_t^{\text{simple}} + C \quad (39)$$

其中 C 是一个不依赖于 θ 的常数。

代码实现中, `p_losses` 函数计算了上述的简单损失:

```
1 def p_losses(denoise_model, x_start, t, noise=None, loss_type="l1"):
2     if noise is None:
3         noise = torch.randn_like(x_start)
4
5     x_noisy = q_sample(x_start=x_start, t=t, noise=noise)
6     predicted_noise = denoise_model(x_noisy, t) # denoise_model is U-Net
7
8     if loss_type == 'l1':
9         loss = F.l1_loss(noise, predicted_noise)
10    elif loss_type == 'l2':
11        loss = F.mse_loss(noise, predicted_noise)
12    elif loss_type == "huber":
13        loss = F.smooth_l1_loss(noise, predicted_noise)
```

```

14     else:
15         raise NotImplementedError()
16
17     return loss

```

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
      $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$ 
6: until converged

```

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 

```

图 2-4 DDPM 中的训练和采样算法

2.2 理论联系与参数化改进

2.2.1 与随机梯度朗之万动力学 (Langevin Dynamics) 的联系

朗之万动力学是物理学中的一个概念，用于统计模拟分子系统。结合随机梯度下降，随机梯度朗之万动力学 (Welling & Teh 2011) 可以仅使用梯度 $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ 在马尔可夫更新链中从概率密度 $p(\mathbf{x})$ 生成样本：

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \frac{\delta}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}_{t-1}) + \sqrt{\delta} \epsilon_t, \quad \text{where } \epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (40)$$

其中 δ 是步长。当 $T \rightarrow \infty, \epsilon \rightarrow 0$ 时， $\mathbf{x}_T$ 等价于真实概率密度 $p(\mathbf{x})$ 。相比标准 SGD，它通过注入高斯噪声来避免陷入局部极小值。

2.2.2 与噪声条件分数网络 (NCSN) 的联系

Song & Ermon (2019) 提出了基于分数的生成模型。该模型通过分数匹配 (Score Matching) 估计数据分布的梯度 (即分数 $\nabla_{\mathbf{x}} \log q(\mathbf{x})$)，并利用朗之万动力学生成样本。为了解决高维数据在低维流形上分布导致的分数估计不准确问题，他们提出添加不同级别的高斯噪声来扰动数据，并训练一个噪声条件分数网络 $s_{\theta}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log q(\mathbf{x})$ 。

如果我们使用扩散过程的标记，分数近似为 $s_{\theta}(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$ 。回忆 $q(\mathbf{x}_t | \mathbf{x}_0) \sim \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$ ，因此：

$$s_{\theta}(\mathbf{x}_t, t) \approx -\frac{\epsilon_{\theta}(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}} \quad (41)$$

这表明预测噪声 ϵ_{θ} 本质上是在预测分数的缩放版本。

2.2.3 β_t 的参数化 (Parameterization of β_t)

Ho et al. (2020) 使用线性增加的常数序列作为方差调度，从 $\beta_1 = 10^{-4}$ 到 $\beta_T = 0.02$ 。Nichol & Dhariwal (2021) 提出了一种基于余弦的方差调度，以获得更好的对数似

然（NLL）。这种调度函数在训练过程中提供近乎线性的下降，而在 $t = 0$ 和 $t = T$ 附近变化微妙。

$$\beta_t = \text{clip}\left(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}, 0.999\right), \quad \bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2 \quad (42)$$

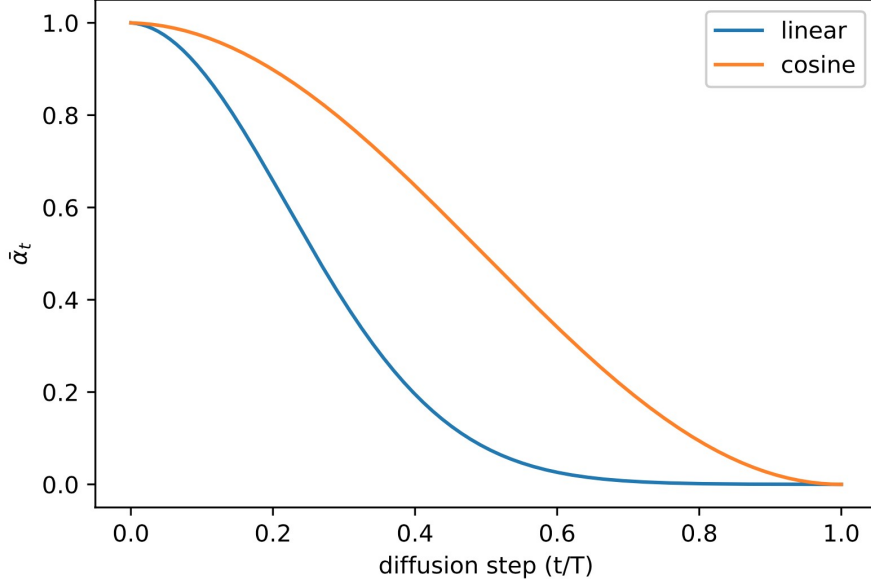


图 2-5 训练期间 β_t 的线性调度与余弦调度的比较

2.2.4 逆向方差 Σ_θ 的参数化

Ho et al. (2020) 将 β_t 设为常数，并设定 $\Sigma_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I}$ ，其中 σ_t 未学习。Nichol & Dhariwal (2021) 提出将 $\Sigma_\theta(\mathbf{x}_t, t)$ 学习为 β_t 和 $\tilde{\beta}_t$ 之间的插值，模型输出一个混合向量 v ：

$$\Sigma_\theta(\mathbf{x}_t, t) = \exp(v \log \beta_t + (1 - v) \log \tilde{\beta}_t) \quad (43)$$

这需要引入混合目标函数 $L_{\text{hybrid}} = L_{\text{simple}} + \lambda L_{\text{VLB}}$ 来指导 Σ_θ 的学习。

Model	ImageNet	CIFAR
Glow (Kingma & Dhariwal, 2018)	3.81	3.35
Flow++ (Ho et al., 2019)	3.69	3.08
PixelCNN (van den Oord et al., 2016c)	3.57	3.14
SPN (Menick & Kalchbrenner, 2018)	3.52	-
NVAE (Vahdat & Kautz, 2020)	-	2.91
Very Deep VAE (Child, 2020)	3.52	2.87
PixelSNAIL (Chen et al., 2018)	3.52	2.85
Image Transformer (Parmar et al., 2018)	3.48	2.90
Sparse Transformer (Child et al., 2019)	3.44	2.80
Routing Transformer (Roy et al., 2020)	3.43	-
DDPM (Ho et al., 2020)	3.77	3.70
DDPM (cont flow) (Song et al., 2020b)	-	2.99
Improved DDPM (ours)	3.53	2.94

图 2-6 改进版 DDPM 与其他生成模型的负对数似然 (NLL) 比较

2.3 条件生成 (Conditioned Generation)

虽然在 ImageNet 数据集等带有条件信息的图像上训练生成模型，但通常会根据类别标签或一段描述性文本来生成样本。

2.3.1 分类器引导扩散 (Classifier Guided Diffusion)

为了将类别信息显式地合并到扩散过程中，Dhariwal & Nichol (2021) 在噪声图像 $\mathbf{x}_t$ 上训练了一个分类器 $f_\phi(y|\mathbf{x}_t, t)$ ，并使用梯度 $\nabla_{\mathbf{x}_t} \log f_\phi(y|\mathbf{x}_t, t)$ 来引导扩散采样过程朝向条件信息 y （例如目标类别标签）移动。回顾 $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t)$ ，我们可以将联合分布 $q(\mathbf{x}_t, y)$ 的分数函数写为：

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t, y) &= \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log q(y|\mathbf{x}_t) \\ &\approx -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t) + \nabla_{\mathbf{x}_t} \log f_\phi(y|\mathbf{x}_t) \\ &= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \left(\epsilon_\theta(\mathbf{x}_t, t) - \sqrt{1-\bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log f_\phi(y|\mathbf{x}_t) \right)\end{aligned}\quad (44)$$

由此产生的扰动预测器 $\bar{\epsilon}_\theta$ 形式如下：

$$\bar{\epsilon}_\theta(\mathbf{x}_t, t) = \epsilon_\theta(\mathbf{x}_t, t) - \sqrt{1-\bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log f_\phi(y|\mathbf{x}_t) \quad (45)$$

为了控制分类器引导的强度，我们可以向 delta 部分添加权重 w ：

$$\bar{\epsilon}_\theta(\mathbf{x}_t, t) = \epsilon_\theta(\mathbf{x}_t, t) - \sqrt{1-\bar{\alpha}_t} w \nabla_{\mathbf{x}_t} \log f_\phi(y|\mathbf{x}_t) \quad (46)$$

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $f_\phi(y|x_t)$, and gradient scale s .

```

Input: class label  $y$ , gradient scale  $s$ 
 $x_T \leftarrow$  sample from  $\mathcal{N}(0, \mathbf{I})$ 
for all  $t$  from  $T$  to 1 do
     $\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$ 
     $x_{t-1} \leftarrow$  sample from  $\mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log f_\phi(y|x_t), \Sigma)$ 
end for
return  $x_0$ 
```

Algorithm 2 Classifier guided DDIM sampling, given a diffusion model $\epsilon_\theta(x_t)$, classifier $f_\phi(y|x_t)$, and gradient scale s .

```

Input: class label  $y$ , gradient scale  $s$ 
 $x_T \leftarrow$  sample from  $\mathcal{N}(0, \mathbf{I})$ 
for all  $t$  from  $T$  to 1 do
     $\hat{\epsilon} \leftarrow \epsilon_\theta(x_t) - \sqrt{1-\bar{\alpha}_t} \nabla_{x_t} \log f_\phi(y|x_t)$ 
     $x_{t-1} \leftarrow \sqrt{\bar{\alpha}_{t-1}} \left( \frac{x_t - \sqrt{1-\bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1-\bar{\alpha}_{t-1}} \hat{\epsilon}$ 
end for
return  $x_0$ 
```

图 2-7 使用分类器引导来进行条件生成的算法 (DDPM 和 DDIM)

2.3.2 无分类器引导 (Classifier-Free Guidance)

即使没有独立的分类器 f_ϕ ，也可以通过结合条件和无条件扩散模型的分数的来运行条件扩散步骤 (Ho & Salimans, 2021)。具体来说，我们在成对数据 $(\mathbf{x}, y)$ 上训练条件扩散模型 $p_\theta(\mathbf{x}|y)$ ，其中条件信息 y 会随机被丢弃，以便模型也学会无条件地生成图像，即 $\epsilon_\theta(\mathbf{x}_t, t) = \epsilon_\theta(\mathbf{x}_t, t, y = \emptyset)$ 。

隐式分类器的梯度可以用条件和无条件分数估计器来表示：

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t) &= \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|y) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \\ &= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} (\epsilon_\theta(\mathbf{x}_t, t, y) - \epsilon_\theta(\mathbf{x}_t, t))\end{aligned}\quad (47)$$

将其代入分类器引导的分数公式中，分数便不再依赖单独的分类器：

$$\begin{aligned}\bar{\epsilon}_\theta(\mathbf{x}_t, t, y) &= \epsilon_\theta(\mathbf{x}_t, t, y) - \sqrt{1-\bar{\alpha}_t} w \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t) \\ &= \epsilon_\theta(\mathbf{x}_t, t, y) + w (\epsilon_\theta(\mathbf{x}_t, t, y) - \epsilon_\theta(\mathbf{x}_t, t)) \\ &= (w+1)\epsilon_\theta(\mathbf{x}_t, t, y) - w\epsilon_\theta(\mathbf{x}_t, t)\end{aligned}\quad (48)$$

实验表明，无分类器引导可以在 FID（图像保真度）和 IS（多样性）之间取得良好的平衡。GLIDE (Nichol, Dhariwal & Ramesh, et al. 2022) 发现无分类器引导优于 CLIP 引导。

2.4 加速扩散模型 (Speed up Diffusion Models)

从 DDPM 生成样本非常慢，因为 T 可能高达 1000 步。一重简单的加速方法是运行跨步采样（Strided Sampling），例如每隔 $\lceil T/S \rceil$ 步进行一次更新。

另一种方法重写 $q_\sigma(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ ，使其由所需的标准差 σ_t 参数化：

$$\begin{aligned}\mathbf{x}_{t-1} &= \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_{t-1} \\ &= \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\epsilon_t + \sigma_t\epsilon \\ &= \sqrt{\bar{\alpha}_{t-1}}\left(\frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta^{(t)}(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}}\right) + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\epsilon_\theta^{(t)}(\mathbf{x}_t) + \sigma_t\epsilon\end{aligned}\quad (49)$$

这样我们得到：

$$q_\sigma(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\left(\frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta^{(t)}(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}}\right) + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\epsilon_\theta^{(t)}(\mathbf{x}_t), \sigma_t^2\mathbf{I}\right)\quad (50)$$

回忆 $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \cdot \beta_t$ ，令 $\sigma_t^2 = \eta \cdot \tilde{\beta}_t$ ，我们可以通过超参数 $\eta \in \mathbb{R}^+$ 控制采样随机性。

Note 2.4 (章节分工). 本节先给出“条件生成”的基本概念与常用做法；更系统的条件控制（Classifier Guidance/CFG 的推导与采样对接、ControlNet/InstantID/LoRA 的组合方式等）统一放在后文的条件生成与控制章节中。

2.4.1 DDIM (Denoising Diffusion Implicit Models)

当 $\eta = 0$ 时，采样过程变得确定性，此时模型称为去噪扩散隐模型 (DDIM)。DDIM 更新步骤（针对加速轨迹中的 $s < t$ ）：

$$q_{\sigma, s < t}(\mathbf{x}_s | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_s; \sqrt{\bar{\alpha}_s} \left(\frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}^{(t)}(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_s - \sigma_t^2 \epsilon_{\theta}^{(t)}(\mathbf{x}_t)}, \sigma_t^2 \mathbf{I}\right) \quad (51)$$

DDIM 具有一致性属性，即以相同的隐变量为条件的多个样本应具有相似的高级特征。这也是因为 DDIM 可以在隐变量中进行语义上有意义的插值。

2.4.2 渐进式蒸馏 (Progressive Distillation)

Salimans & Ho (2022) 提出了一种将训练好的确定性采样器蒸馏成步数减半的新模型的方法。在每次蒸馏迭代中，我们可以将采样步骤减半。学生模型初始化自教师模型，并朝着匹配 2 个教师 DDIM 步骤的目标进行去噪（即算法中的 $z_{t''}$ ），而不是使用原始样本 $\mathbf{x}_0$ 作为去噪目标。

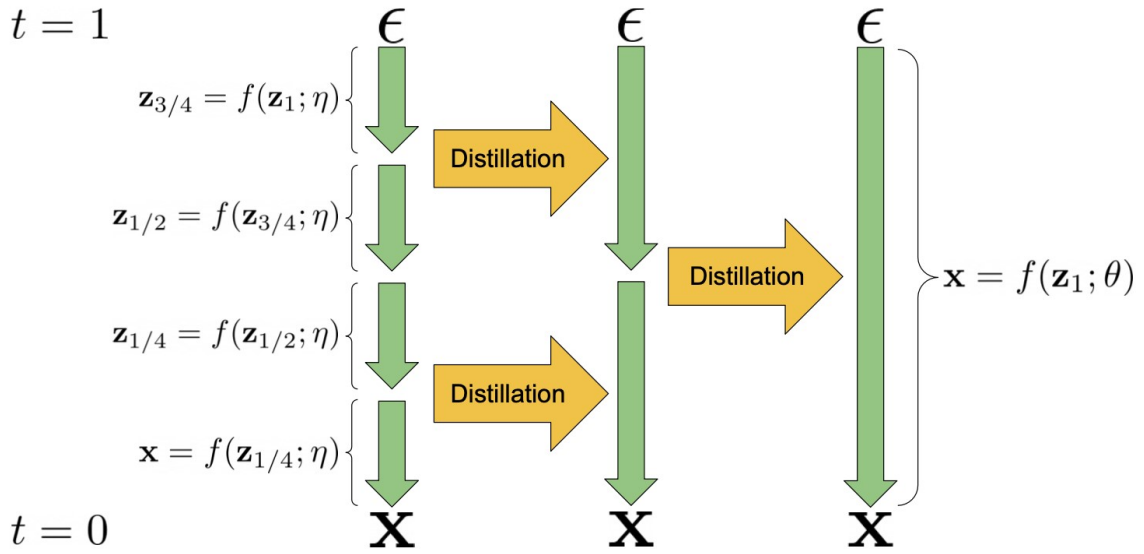


图 2-8 渐进式蒸馏可以在每次迭代中将扩散采样步骤减半

2.4.3 一致性模型 (Consistency Models)

Song et al. (2023) 提出的一致性模型旨在学习将扩散采样轨迹上的任何中间噪声数据点 $\mathbf{x}_t$ 直接映射回其原点 $\mathbf{x}_0$ 。如名称所示，它具有自一致性。

一致性模型定义为 $f_{\theta} : (\mathbf{x}_t, t) \mapsto \mathbf{x}_{\epsilon}$ ，使得对同一轨迹上的所有点输出相同。它可以被参数化为：

$$f_{\theta}(\mathbf{x}, t) = c_{\text{skip}}(t)\mathbf{x} + c_{\text{out}}(t)F_{\theta}(\mathbf{x}, t) \quad (52)$$

该论文介绍了两种训练一致性模型的方法：

1. 一致性蒸馏 (Consistency Distillation, CD): 将预训练的扩散模型蒸馏为一致性模型。损失函数为:

$$\mathcal{L}_{CD}^N(\theta, \theta^-; \phi) = \mathbb{E}[\lambda(t_n) d(f_\theta(\mathbf{x}_{t_{n+1}}, t_{n+1}), f_{\theta^-}(\mathbf{x}_{t_n}^\phi, t_n))] \quad (53)$$

其中 θ^- 是 θ 的 EMA 版本。

2. 一致性训练 (Consistency Training, CT): 独立训练一致性模型。损失函数为:

$$\mathcal{L}_{CT}^N(\theta, \theta^-; \phi) = \mathbb{E}[\lambda(t_n) d(f_\theta(\mathbf{x} + t_{n+1}\mathbf{z}, t_{n+1}), f_{\theta^-}(\mathbf{x} + t_n\mathbf{z}, t_n))] \quad (54)$$

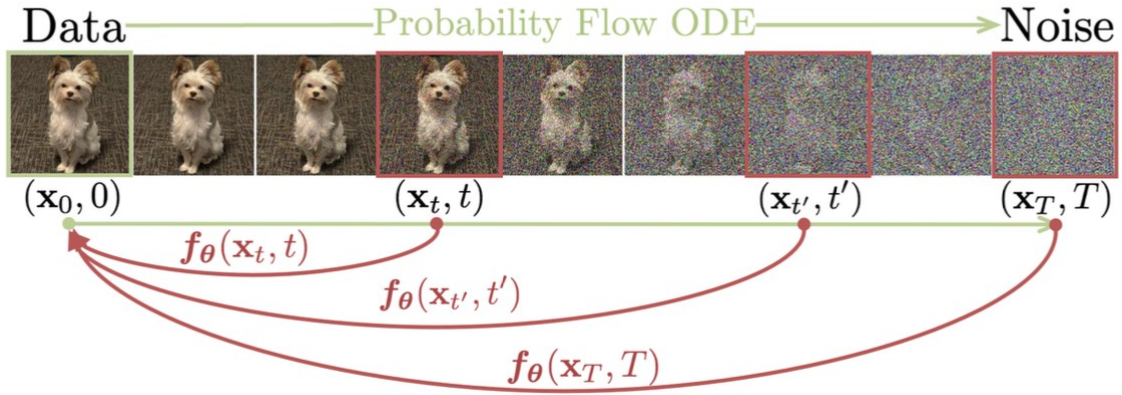


图 2-9 一致性模型学习将轨迹上的任何数据点映射回其原点

2.5 潜在扩散模型 (Latent Diffusion Models, LDM)

传统的扩散模型（如 DDPM）直接在像素空间（Pixel Space）操作。由于图像维度通常很高（例如 $512 \times 512 \times 3$ ），这导致训练和推理的计算成本极高。Rombach et al. (2022) 提出的潜在扩散模型（LDM，即 Stable Diffusion 的基础）基于一个核心观察：图像包含两类信息——感知细节（高频与局部特征）和语义信息（宏观结构与概念）。前者可以通过通用的自动编码器高效压缩，而后者才是扩散模型需要重点学习生成的。

因此，LDM 将生成过程分离为两个阶段：

1. 感知压缩 (Perceptual Compression): 训练一个自编码器，将高维像素空间 $\mathcal{X}$ 压缩到低维潜在空间 $\mathcal{Z}$ 。
2. 潜在扩散 (Latent Diffusion): 在低维潜在空间 $\mathcal{Z}$ 中训练扩散模型来生成潜在向量。

2.5.1 第一阶段：感知压缩 (Autoencoder)

引入一个预训练的感知自动编码器，包含编码器 $\mathcal{E}$ 和解码器 $\mathcal{D}$ 。

- **编码：**给定图像 $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ ，编码器将其映射为潜在表示 $\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{h \times w \times c}$ 。通常下采样因子 $f = H/h = W/w$ 为 2^m （如 $f = 8$ ）。
- **解码：**解码器将潜在向量重建为图像 $\tilde{\mathbf{x}} = \mathcal{D}(\mathbf{z}) \approx \mathbf{x}$ 。
- **正则化：**为了避免潜在空间出现高方差或离散化问题，通常使用 KL 正则化（类似 VAE）或 VQ 正则化（类似 VQ-GAN）。

Note 2.5 (优势). 相比于像素空间， $\mathbf{z}$ 的维度大大降低。例如 $f = 8$ 时，数据体积减少为原来的 $1/64$ 。这使得扩散模型的训练极其高效，且允许我们在有限的计算资源上生成高分辨率图像。

2.5.2 第二阶段：潜在扩散 (Diffusion in Latent Space)

现在的扩散过程不再直接作用于 $\mathbf{x}$ ，而是作用于潜在向量 $\mathbf{z}$ 。

1. **前向过程：**向 $\mathbf{z}_0 = \mathcal{E}(\mathbf{x})$ 添加噪声得到 $\mathbf{z}_t$ 。

$$\mathbf{z}_t = \sqrt{\bar{\alpha}_t} \mathcal{E}(\mathbf{x}) + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon} \quad (55)$$

2. **逆向过程：**训练一个去噪网络 $\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, y)$ ，预测添加到 $\mathbf{z}_t$ 上的噪声。

最终的优化目标函数 L_{LDM} 修改为：

$$L_{LDM} = \mathbb{E}_{\mathcal{E}(\mathbf{x}), \boldsymbol{\epsilon} \sim \mathcal{N}(0,1), t} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, \tau_\theta(y))\|_2^2] \quad (56)$$

其中 $\tau_\theta(y)$ 是条件输入（如文本、语义图）的编码表示。

2.5.3 条件机制 (Conditioning Mechanism)

LDM 通过交叉注意力机制 (Cross-Attention) 灵活地注入条件信息 y 。去噪网络通常采用 U-Net 架构。条件 y （如文本提示“A cute cat”）首先通过特定领域的编码器 τ_θ （如 CLIP Text Encoder）转换为序列向量 $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$ 。

然后，该条件向量通过交叉注意力层作用于 U-Net 的中间层特征 $\varphi_i(z_t) \in \mathbb{R}^{N \times d_\epsilon^i}$ ：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) \cdot V \quad (57)$$

具体的投影矩阵定义为：

$$Q = W_Q^{(i)} \cdot \varphi_i(z_t), \quad K = W_K^{(i)} \cdot \tau_\theta(y), \quad V = W_V^{(i)} \cdot \tau_\theta(y) \quad (58)$$

这里：

- Q (Query) 来自图像当前的潜在特征（Visual Features）。
- K (Key) 和 V (Value) 来自条件嵌入（Text Features）。

这使得模型能够根据文本提示关注图像中的特定区域并进行生成。

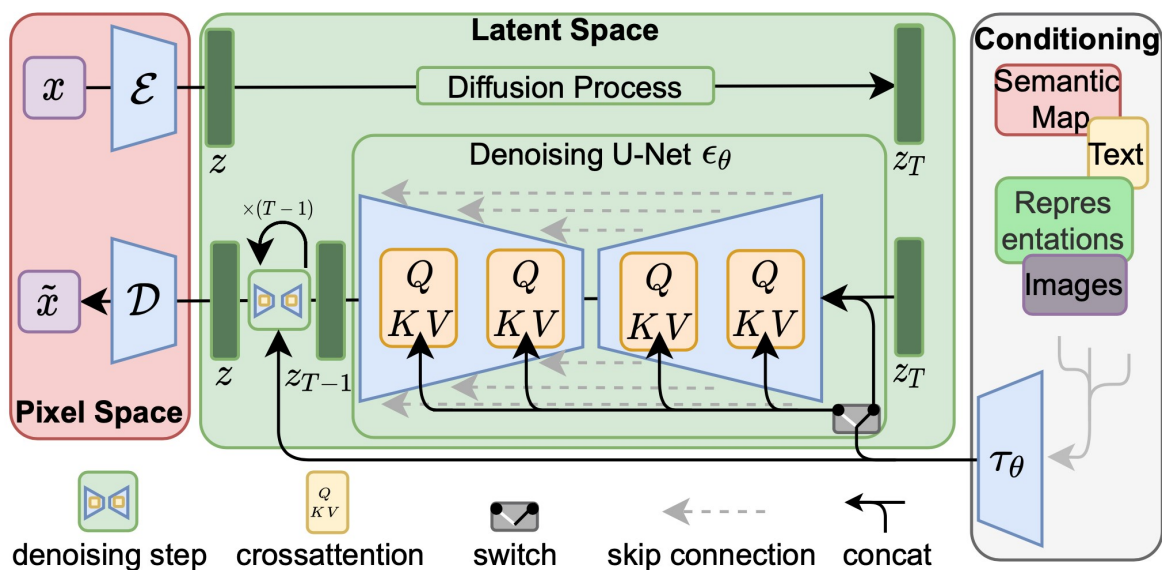


图 2-10 潜在扩散模型 (LDM) 的架构：左侧为感知压缩（Autoencoder），中间为潜在空间的扩散过程，右侧为通过交叉注意力注入条件信息。

Note 2.6 (章节分工). 本小节在第二章只说明“条件如何作为接口注入到去噪网络”（以交叉注意力为代表）；更完整的条件控制推导与实践（CFG/ControlNet/LoRA/InstantID 以及它们的组合）统一放在后文条件生成与控制章节。

2.6 扩大生成分辨率与质量

为了生成高分辨率的高质量图像，Ho et al. (2021) 提出使用多个扩散模型的级联管道。管道模型之间的噪声条件增强（Noise Conditioning Augmentation）对于最终图像质量至关重要。

2.6.1 级联扩散模型 (Cascaded Diffusion Models)

级联管道包括一个生成低分辨率图像的基础模型，以及随后的一个或多个超分辨率（Super-Resolution）扩散模型。研究发现，最有效的调节噪声是在低分辨率下施加高斯噪声，在高分辨率下施加高斯模糊。

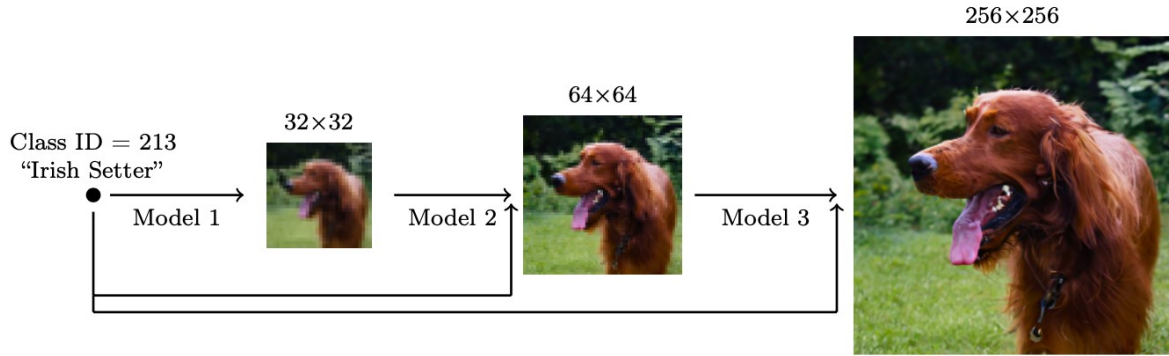


图 2-11 多个扩散模型在增加分辨率时的级联管道

2.6.2 unCLIP (DALL · E 2)

unCLIP (Ramesh et al. 2022) 是一种利用 CLIP 文本编码器的两阶段扩散模型。它并行学习两个模型：

1. 先验模型 (Prior) $P(c^i|y)$ ：给定文本 y ，输出 CLIP 图像嵌入 c^i 。
2. 解码器 (Decoder) $P(x|c^i, [y])$ ：给定 CLIP 图像嵌入 c^i （以及可选的原始文本 y ），生成图像 x 。

这两个模型通过贝叶斯规则实现条件生成：

$$P(x|y) = \underbrace{P(x, c^i|y)}_{c^i \text{ is deterministic given } x} = P(x|c^i, y)P(c^i|y) \quad (59)$$

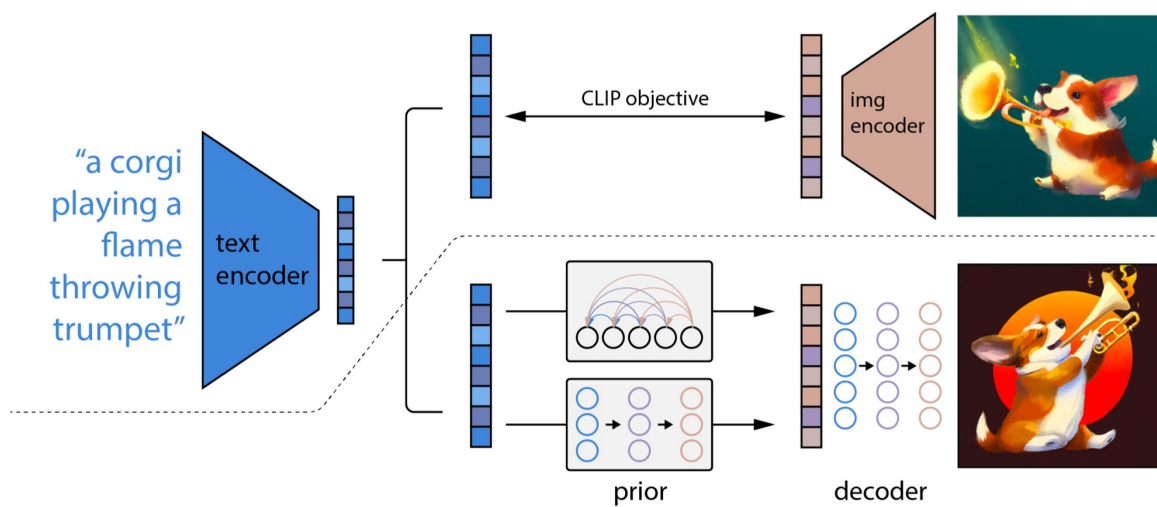


图 2-12 unCLIP 的架构

2.6.3 Imagen

Imagen (Saharia et al. 2022) 使用预训练的大型语言模型（即冻结的 T5-XXL 文本编码器）对文本进行编码。研究发现，扩大文本编码器的规模比扩大 U-Net 的规模对图像质量更重要。

2.7 模型架构 (Model Architecture)

扩散模型的核心是噪声预测网络 $\epsilon_{\theta}(\mathbf{x}_t, t, y)$ 。该网络必须满足两个条件：

1. **输入输出同维**：输入是含噪图像 $\mathbf{x}_t$ ，输出是预测的噪声 ϵ （或去噪后的图像），两者维度相同（如 $H \times W \times C$ ）。
2. **条件注入**：必须能够接收时间步 t 和其他条件 y （文本、类别等）作为输入。

目前主流的骨干网络架构主要有两种：U-Net 和 Transformer (DiT)。此外，为了实现精细的空间控制，ControlNet 也是一种重要的架构变体。

符号约定（避免前后混乱） 在第二章中我们常用 y 表示“文本/类别等条件”。在后文更一般的条件控制里会把所有条件统一写成 $\mathbf{c}$ ，并在多模态时写作 $\mathbf{c} = (\mathbf{c}_{text}, \mathbf{c}_{img}, \dots)$ 。因此可以把本章的 y 理解为 $\mathbf{c}_{text}$ 的简写。

2.7.1 U-Net 架构

U-Net (Ronneberger et al., 2015) 最初用于医学图像分割，因其强大的多尺度特征融合能力而被引入扩散模型（最早在 DDPM 中使用）。

1. 总体结构

U-Net 由三个部分组成：

- **下采样路径 (Encoder)**：通过一系列卷积层和池化层（或步幅卷积）逐步减小特征图的空间分辨率，同时增加通道数。这部分负责提取图像的高级语义特征。
- **瓶颈层 (Bottleneck)**：在分辨率最低处（如 8×8 ）进行处理，通常包含自注意力机制以捕捉全局上下文。
- **上采样路径 (Decoder)**：通过反卷积或插值逐步恢复空间分辨率。
- **跳跃连接 (Skip Connections)**：将 Encoder 中对应层的特征图直接拼接到 Decoder 中。这对扩散模型至关重要，因为预测的噪声通常包含高频细节，通过跳跃连接可以直接传递低层级的纹理信息，避免信息在深层网络中丢失。

2. 时间步嵌入 (Time Embedding)

为了让网络知道当前的去噪进度，时间步 t 被映射为一个向量（类似于 Transformer 的位置编码，通常使用正弦位置编码 Sinusoidal Embedding）。该向量通过 MLP 处理后，被

注入到 U-Net 的每一个残差块（ResBlock）中，通常采用 **相加 (Addition)** 或 **自适应归一化 (AdaGN/FiLM)** 的方式：

$$\text{AdaGN}(h, t) = y_s(t) \cdot \frac{h - \mu}{\sigma} + y_b(t) \quad (60)$$

其中 $y_s(t)$ 和 $y_b(t)$ 是由时间嵌入生成的缩放和平移系数。

3. 交叉注意力 (Cross-Attention)

为了注入文本或其他条件 y ，U-Net 在各个分辨率层级均插入了空间变换器（Spatial Transformer）模块，主要包含交叉注意力层：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (61)$$

其中 Q 来自图像特征， K, V 来自条件编码（如 CLIP Text Embedding）。

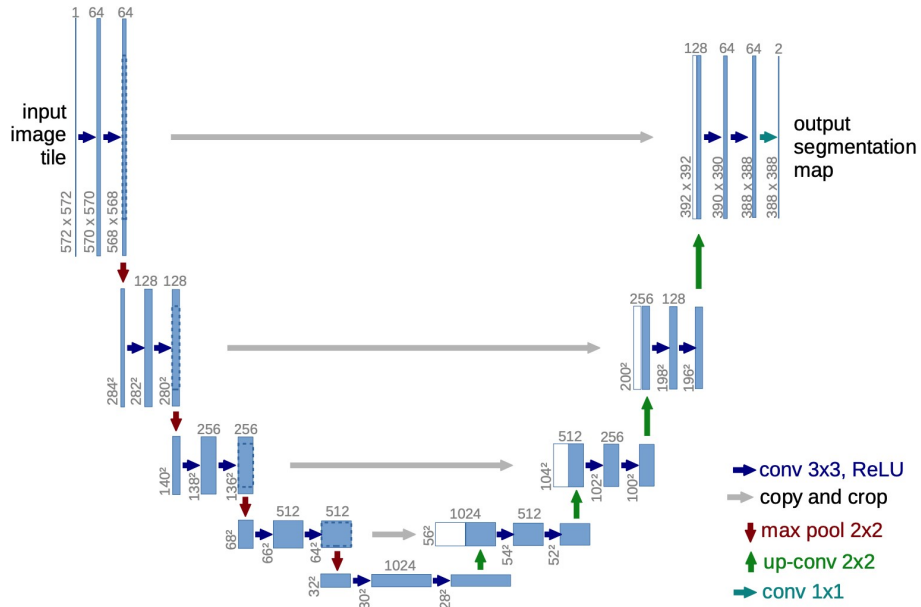


图 2-13 U-Net 架构

2.7.2 Diffusion Transformer (DiT)

虽然 U-Net 在早期扩散模型中占据统治地位，但基于 Transformer 的架构（如 DiT, Peebles & Xie, 2023）正在成为新一代主流（如 Sora, Stable Diffusion 3）。

1. Patchify (类似 ViT)

DiT 抛弃了 U-Net 的卷积归纳偏置。它首先将输入的含噪图像（或潜在特征 $\mathbf{z}$ ）划分为一个个固定大小的补丁（Patches，例如 4×4 或 2×2 ）。这些 Patches 被展平并线性投影为序列向量（Tokens），输入到标准的 Transformer 块中。

2. 架构优势

- **可扩展性 (Scalability):** DiT 证明了扩散模型的性能随着计算量 (Gflops) 和参数量的增加而呈现幂律 (Scaling Law) 提升。
- **全局交互:** 相比于卷积的局部感受野, Self-Attention 机制允许每个 Patch 从第一层开始就与全图其他 Patch 交互, 更利于处理长距离依赖。

3. 自适应层归一化 (adaLN-Zero)

DiT 处理时间步 t 和类别标签 c 的方式非常简洁。它在每个 Transformer 块之前回归出缩放移位参数 γ, β 以及维度缩放参数 α , 直接调节层归一化 (Layer Norm):

$$\text{adaLN}(x, c, t) = \gamma(c, t) \cdot \text{LayerNorm}(x) + \beta(c, t) \quad (62)$$

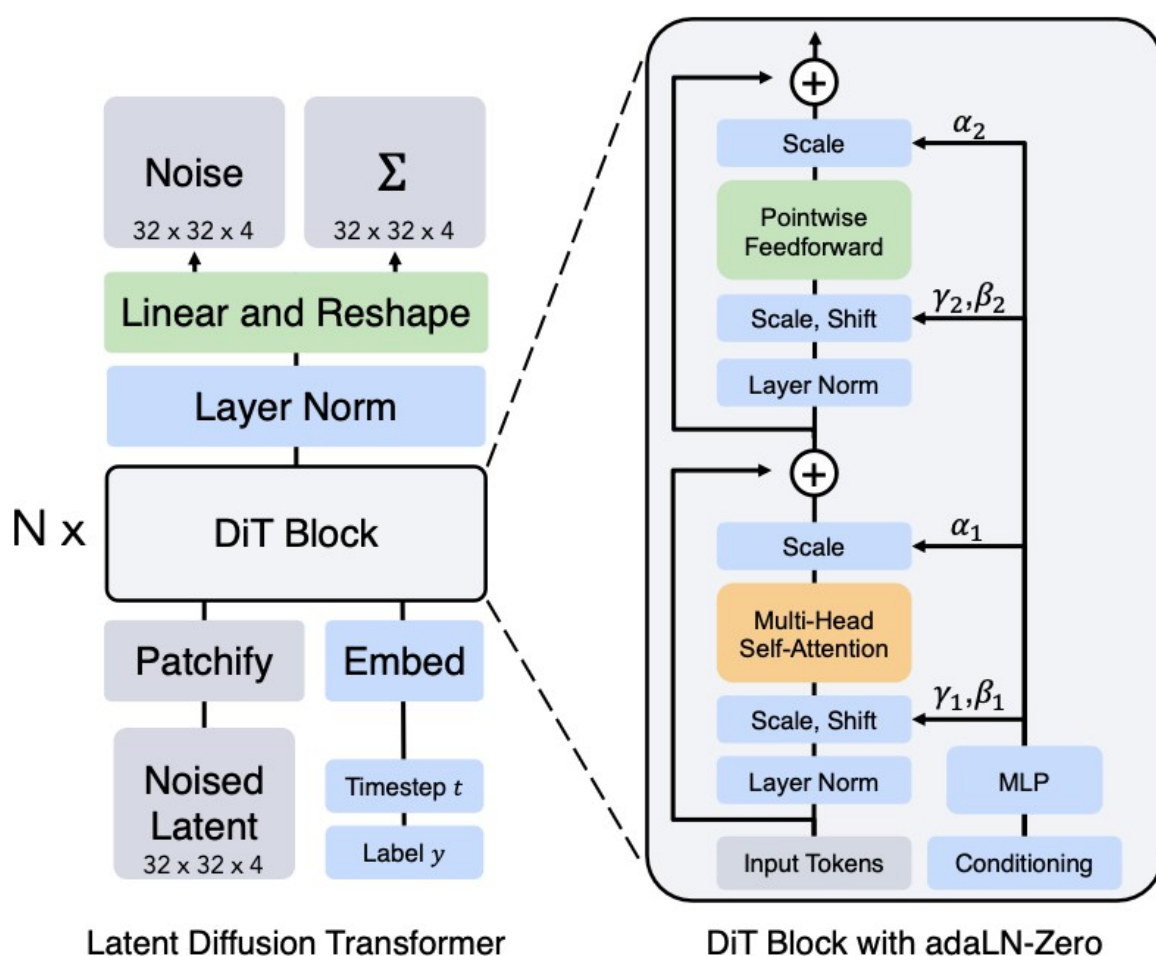


图 2-14 Diffusion Transformer (DiT) 架构

2.7.3 ControlNet

ControlNet (Zhang et al., 2023) 是一种用于向预训练扩散模型添加空间控制条件 (如 Canny 边缘、姿态骨架、深度图) 的神经网络结构。它的核心设计理念是“不破坏原始模型的生成能力”。

1. 零卷积 (Zero Convolution)

这是 ControlNet 的核心组件。它是一个 1×1 卷积层，其权重和偏置初始化为零。

$$y = \text{ZeroConv}(x) = \mathbf{0} \cdot x + \mathbf{0} = \mathbf{0} \quad (63)$$

这保证了在训练初始阶段，ControlNet 分支的输出为零，对主模型的干扰为零，就像 ControlNet 不存在一样。随着训练进行，权重逐渐更新，控制信号才开始生效。

2. 锁定副本与可训练副本 (Locked Copy & Trainable Copy)

ControlNet 并不直接微调整个 U-Net，而是：

- **锁定副本：**保留原始预训练 U-Net 的编码器部分，参数冻结 (Locked)，确保原有的图像生成能力不退化。
- **可训练副本：**克隆一份 U-Net 的编码器结构，但这部分参数是可训练的。它接收额外的控制条件（如边缘图 c_{img} ）作为输入。

这两个分支的特征图通过零卷积层相加，注入到原始 U-Net 的解码器部分：

$$h_{decoder} = h_{locked} + \text{ZeroConv}(h_{trainable}(x + c_{img})) \quad (64)$$

一种更“整体”的写法是把它看成对任意网络块 $F_\theta(\cdot)$ 的改造（冻结 θ ，训练副本 θ_c ，并用零卷积注入控制条件 $\mathbf{c}$ ）：

$$y_c = F_\theta(\mathbf{x}) + Z_{\theta_{z2}} \left(F_{\theta_c}(\mathbf{x} + Z_{\theta_{z1}}(\mathbf{c})) \right) \quad (65)$$

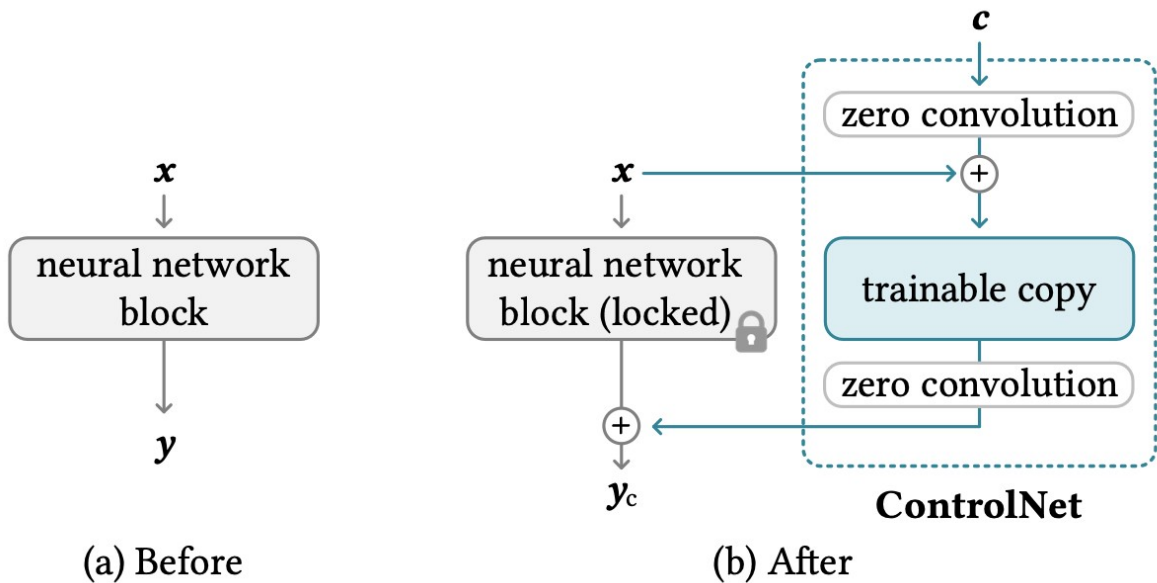


图 2-15 ControlNet 架构

Note 2.7 (阅读提示). 本节只给出 U-Net/DiT/ControlNet 的架构要点，以便建立“扩散 = 过程 + 去噪网络”的主线；更细的条件控制推导与工程组合方式统一见后文条件生成与控制章节。

交叉注意力公式（完整写法） 设某一层图像特征（或潜空间特征）为 $\varphi_i(\mathbf{x}_t)$ ，则

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad (66)$$

并且

$$Q = W_Q \varphi_i(\mathbf{x}_t), \quad K = W_K \mathbf{c}_{text}, \quad V = W_V \mathbf{c}_{text}. \quad (67)$$

这等价于：每一步去噪时，图像特征“询问”文本条件中与当前局部最相关的 token，从而把语义约束注入生成过程。

空提示与负面提示 (Negative Prompt) 后文的 CFG 会用到“无条件”分支。最常见的是空提示 $\emptyset$ （空字符串）作为无条件文本： $\mathbf{c}_{uncond} = \tau(\emptyset)$ 。在实际系统里也常用负面提示 y^- 作为无条件分支（例如“low quality, blurry, ...”）：

$$\mathbf{c}_{uncond} = \tau(y^-), \quad \mathbf{c}_{cond} = \tau(y^+). \quad (68)$$

2.8 基于分类器的引导 (Classifier Guidance)

在条件生成里，一个非常关键的等式是：

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{c}) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{c} | \mathbf{x}_t). \quad (69)$$

它说明：条件 score 等于无条件 score 加上一个“把样本往条件方向推”的梯度项。

若我们有一个额外训练的分类器/匹配模型 $p_\phi(\mathbf{c} | \mathbf{x}_t)$ ，则可以用它的梯度来修正去噪方向（见^[1]）：

$$\mathbf{s}_{guided}(\mathbf{x}_t, t, \mathbf{c}) = \mathbf{s}_\theta(\mathbf{x}_t, t) + w \nabla_{\mathbf{x}_t} \log p_\phi(\mathbf{c} | \mathbf{x}_t), \quad (70)$$

其中 $\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log p_\theta(\mathbf{x}_t)$ 是扩散模型自身提供的无条件 score， w 控制引导强度。

该方法的代价是：需要额外的分类器，并且每一步采样都要对 $\log p_\phi$ 求梯度（更慢、更复杂）。因此在文本到图像系统里，主流逐渐转向下一节的 CFG。

2.9 无分类器引导 (Classifier-Free Guidance, CFG)

CFG（Ho & Salimans, 2022，见^[2]）的核心点是：不再训练额外分类器，而是让同一个去噪网络同时学会“有条件/无条件”两种行为。

2.9.1 训练：条件 dropout 让模型学会无条件

训练时以概率 p_{drop} 把条件替换为空条件 $\emptyset$ ：

$$\mathbf{c} = \begin{cases} \emptyset, & \text{with prob. } p_{drop} \\ y, & \text{with prob. } 1 - p_{drop} \end{cases} \quad (71)$$

于是同一个网络可以输出两种噪声预测：

$$\epsilon_{cond} = \epsilon_\theta(\mathbf{x}_t, t, y), \quad \epsilon_{uncond} = \epsilon_\theta(\mathbf{x}_t, t, \emptyset). \quad (72)$$

2.9.2 推理：CFG 的“公式原版”与等价写法

CFG 在推理阶段把两次前向的结果做线性组合：

$$\tilde{\epsilon}_\theta(\mathbf{x}_t, t, y) = \epsilon_{uncond} + w(\epsilon_{cond} - \epsilon_{uncond}). \quad (73)$$

等价地写作：

$$\tilde{\epsilon}_\theta = (1 + w)\epsilon_{cond} - w\epsilon_{uncond}. \quad (74)$$

2.9.3 与采样更新的“对接”

一旦得到 $\tilde{\epsilon}_\theta$ ，就把它当作普通噪声预测塞回去噪更新即可。一个常见写法是：

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \tilde{\epsilon}_\theta \right) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (75)$$

因此，CFG 的本质非常“工程化”：只要替换噪声预测，整个采样过程不需要改任何东西。

引导尺度 w 的含义 当 $w = 0$ 时，退化为无条件生成；当 w 增大时，语义对齐更强，但也更容易出现过饱和、细节怪异、文本过拟合等副作用。

2.10 基于输入图像的条件控制：Img2Img 与 Inpainting

除了“从纯噪声生成”，扩散模型还常做条件编辑。

2.10.1 Img2Img：先把输入图像加噪到 t 再去噪

给定输入图像 $\mathbf{x}_{init}$ ，先采样噪声 $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 并加噪到某一时间步 t ：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_{init} + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (76)$$

然后从 $t \rightarrow 0$ 执行逆向去噪，但在每一步都带条件（文本/风格等）。其中 t 越大，改动越大； t 越小，越接近保留原图结构。

2.10.2 Inpainting：用 mask 锁定已知区域

设 mask $\mathbf{m} \in \{0, 1\}^{H \times W}$ （1 表示“保留/已知区域”）。在采样过程中，常用一个简单而有效的融合规则：

$$\mathbf{x}_t \leftarrow \mathbf{m} \odot \mathbf{x}_t^{known} + (1 - \mathbf{m}) \odot \mathbf{x}_t^{unknown}, \quad (77)$$

其中 $\mathbf{x}_t^{known}$ 可以由原图加噪得到（保证已知区域不漂移）， $\mathbf{x}_t^{unknown}$ 是模型当前一步生成的未知区域。

2.11 精细化空间控制 (ControlNet)

文本条件强在语义弱在几何；要精确控制姿态/边缘/深度等空间结构，常用 ControlNet ([3])。

2.11.1 把 ControlNet 也视作条件 $\mathbf{c}$ 的一部分

我们可以把条件写成多模态组合：

$$\mathbf{c} = (\mathbf{c}_{text}, \mathbf{c}_{img}), \quad (78)$$

其中 $\mathbf{c}_{img}$ 是边缘图/姿态骨架/深度图等。于是噪声预测网络自然扩展为：

$$\epsilon_{\theta}(\mathbf{x}_t, t, \mathbf{c}) = \epsilon_{\theta}(\mathbf{x}_t, t, \mathbf{c}_{text}, \mathbf{c}_{img}). \quad (79)$$

2.11.2 结构：冻结主干 + 可训练副本 + 零卷积

ControlNet 的设计原则是“不破坏原模型”：冻结原始 U-Net 参数 Θ ，并训练一个带参数 Θ_c 的副本，同时用零卷积 $\mathcal{Z}$ 逐步把控制信号注入主干：

$$\mathbf{y}_c = \mathcal{F}(\mathbf{x}; \Theta) + \mathcal{Z}(\mathcal{F}(\mathbf{x} + \mathcal{Z}(\mathbf{c}_{img}; \Theta_{z1}); \Theta_c); \Theta_{z2}). \quad (80)$$

由于初始化时 $\mathcal{Z}(\cdot) = \mathbf{0}$ ，训练初期主干几乎不受扰动；随着训练进行，控制分支逐渐“接管”空间结构。

2.11.3 ControlNet 与 CFG 的组合（最常用写法）

实际系统通常会同时使用文本 CFG 与 ControlNet 空间条件。最常用的做法是：空间条件在两次前向中都保留，只在文本上做 **cond/uncond**：

$$\epsilon_{cond} = \epsilon_{\theta}(\mathbf{x}_t, t, y, \mathbf{c}_{img}), \quad \epsilon_{uncond} = \epsilon_{\theta}(\mathbf{x}_t, t, \emptyset, \mathbf{c}_{img}), \quad (81)$$

再按 CFG 公式得到 $\tilde{\epsilon}_{\theta}$ 并执行采样更新。这样语义与结构约束可以同时增强，并且实现上非常直接。

2.12 身份一致的个性化控制：InstantID

在“条件控制”的谱系里，InstantID ([4]) 解决的是另一类非常实用的问题：给一张参考人脸 (reference face)，在几乎不训练或极少训练的情况下，让生成结果保持同一身份 (identity-preserving)。它可以看作把“身份条件”加入到我们前面统一记号的 $\mathbf{c}$ 里：

$$\mathbf{c} = (\mathbf{c}_{text}, \mathbf{c}_{id}, \mathbf{c}_{kp}), \quad (82)$$

其中 $\mathbf{c}_{id}$ 是身份嵌入 (identity embedding)， $\mathbf{c}_{kp}$ 是与人脸几何结构强相关的条件（如关键点/姿态/轮廓），通常通过 ControlNet 注入。

2.12.1 为什么“只靠文本”很难锁定身份？

文本提示词更像是语义约束（年龄、发型、表情、风格），而“身份”是一个更细粒度、更高维且非常敏感的约束。实践中，如果仅依赖文本 Cross-Attention，模型往往会生成“像某类人”，但无法稳定复现同一个人的面部细节（眼距、五官比例、痣/皱纹等）。因此 InstantID 引入了一个专门的身份向量 $\mathbf{c}_{id}$ 来做条件控制。

2.12.2 步骤 1：从参考人脸提取身份嵌入 $\mathbf{c}_{id}$

给定参考人脸图像 $\mathbf{x}_{ref}$ ，先进行人脸检测与对齐（alignment），再用一个人脸识别模型（例如 ArcFace 类的识别网络）提取身份嵌入：

$$\mathbf{c}_{id} = f_{id}(\mathbf{x}_{ref}) \in \mathbb{R}^{d_{id}}. \quad (83)$$

直观理解： $\mathbf{c}_{id}$ 的设计目标不是“重建这张脸”，而是把身份信息压缩成在不同表情、光照、角度下仍稳定的特征。

2.12.3 步骤 2：把身份嵌入变成可被 Cross-Attention 使用的“条件 token”

为了让 U-Net 的 Cross-Attention 能利用 $\mathbf{c}_{id}$ ，常见做法是把它投影为一组“伪 token”（identity tokens），并与文本 token 拼接（这与很多 image-prompt adapter / IP-Adapter 思路一致）：

$$\mathbf{c}_{id_tok} = P(\mathbf{c}_{id}) \in \mathbb{R}^{M_{id} \times d_{\tau}}, \quad (84)$$

然后把条件序列写为

$$\mathbf{c}_{all} = [\mathbf{c}_{text}; \mathbf{c}_{id_tok}] \in \mathbb{R}^{(M+M_{id}) \times d_{\tau}}. \quad (85)$$

这样 Cross-Attention 的 K, V 就来自同一个序列 $\mathbf{c}_{all}$ ：

$$\mathbf{Q} = W_Q \varphi_i(\mathbf{x}_t), \quad \mathbf{K} = W_K \mathbf{c}_{all}, \quad \mathbf{V} = W_V \mathbf{c}_{all}. \quad (86)$$

它的效果是：去噪时模型既能从文本 token 里取语义约束，也能从身份 token 里取身份约束，从而把“像谁”这一件事变得可控。

2.12.4 步骤 3：用人脸关键点/姿态的 ControlNet 锁定几何结构

仅靠身份 token 仍可能出现面部结构漂移（比如头部姿态、五官位置不稳定）。InstantID 常配合一个几何条件（如 2D/3D 关键点、面部姿态等），并用 ControlNet 形式注入：

$$\epsilon_{\theta}(\mathbf{x}_t, t, \mathbf{c}) = \epsilon_{\theta}(\mathbf{x}_t, t, \mathbf{c}_{all}, \mathbf{c}_{kp}). \quad (87)$$

其中 $\mathbf{c}_{kp}$ 体现“结构该长什么样”，而 $\mathbf{c}_{id}$ 体现“这个人是谁”。两者结合，通常能显著提升身份一致性与结构稳定性。

2.12.5 InstantID 与 CFG 的组合（文本/身份/结构同时存在）

当同时使用 CFG 时，一个常见且有效的组合策略是：无条件分支只去掉文本（或用负面提示），身份与结构条件保持不变，避免“uncond 分支”把身份信息也抹掉：

$$\epsilon_{cond} = \epsilon_{\theta}(\mathbf{x}_t, t, y, \mathbf{c}_{id}, \mathbf{c}_{kp}), \quad \epsilon_{uncond} = \epsilon_{\theta}(\mathbf{x}_t, t, \emptyset, \mathbf{c}_{id}, \mathbf{c}_{kp}). \quad (88)$$

然后仍按 CFG 组合：

$$\tilde{\epsilon}_{\theta} = \epsilon_{uncond} + w(\epsilon_{cond} - \epsilon_{uncond}), \quad (89)$$

并把 $\tilde{\epsilon}_{\theta}$ 代回采样更新式。实践中还可以为身份条件再加一个独立尺度（identity scale），但本质仍是“多条件共同决定去噪方向”。

2.13 参数高效条件适配：LoRA

当我们希望把扩散模型适配到特定风格/人物/领域时，最直接的是全参数微调，但代价高且容易遗忘。LoRA（见^[5]）用“低秩更新”把可训练参数压到极小。

2.13.1 核心公式（含缩放因子）

对任意线性层 $W_0 \in \mathbb{R}^{d \times k}$ ，LoRA 用低秩矩阵近似更新量：

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA, \quad (90)$$

其中 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$, α 是缩放超参。

2.13.2 LoRA 注入位置：为什么常用在 Cross-Attention

在文本到图像模型中，条件信息主要经由 Cross-Attention 进入网络，因此最常见的注入点就是 W_Q, W_K, W_V, W_{out} ：

$$Q = (W_Q + \Delta W_Q) \varphi_i(\mathbf{x}_t), \quad K = (W_K + \Delta W_K) \mathbf{c}_{text}, \quad V = (W_V + \Delta W_V) \mathbf{c}_{text}. \quad (91)$$

训练时冻结 W_0 只训练 A, B ，保存/分发的就是这些小矩阵文件，因此社区里的风格 LoRA 往往只有几 MB。

参考文献

- [1] DHARIWAL P, NICHOL A. Diffusion models beat gans on image synthesis[J]. Advances in Neural Information Processing Systems, 2021.
- [2] HO J, SALIMANS T. Classifier-free diffusion guidance[A]. 2022.
- [3] ZHANG L, RAO A, AGRAWALA M. Adding conditional control to text-to-image diffusion models [A]. 2023.

- [4] Instantid: Zero-shot identity-preserving generation in seconds[EB/OL]. 2024. <https://github.com/InstantID/InstantID>.
- [5] HU E J, SHEN Y, WALLIS P, et al. Lora: Low-rank adaptation of large language models[A]. 2021.
- [6] WENG L. From autoencoder to beta-vae[J/OL]. lilianweng.github.io, 2018. <https://lilianweng.github.io/posts/2018-08-12-vae/>.
- [7] SOHL-DICKSTEIN J, WEISS E, MAHESWARANATHAN N, et al. Deep unsupervised learning using nonequilibrium thermodynamics[J]. International Conference on Machine Learning, 2015.
- [8] SONG Y, ERMON S. Generative modeling by estimating gradients of the data distribution[J]. Advances in Neural Information Processing Systems, 2019.
- [9] HO J, JAIN A, ABBEEL P. Denoising diffusion probabilistic models[J]. Advances in Neural Information Processing Systems, 2020.
- [10] SONG J, MENG C, ERMON S. Denoising diffusion implicit models[A]. 2020.
- [11] NICHOL A, DHARIWAL P, RAMESH A, et al. Glide: Towards photorealistic image generation and editing with text-guided diffusion models[A]. 2021.
- [12] ROMBACH R, BLATTMANN A, LORENZ D, et al. High-resolution image synthesis with latent diffusion models[J]. Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2022.
- [13] RAMESH A, DHARIWAL P, NICHOL A, et al. Hierarchical text-conditional image generation with clip latents[A]. 2022.
- [14] SAHARIA C, CHAN W, SAXENA S, et al. Photorealistic text-to-image diffusion models with deep language understanding[J]. Advances in Neural Information Processing Systems, 2022.
- [15] RONNEBERGER O, FISCHER P, BROX T. U-net: Convolutional networks for biomedical image segmentation[J]. Medical image computing and computer-assisted intervention–MICCAI 2015, 2015.
- [16] PEEBLES W, XIE S. Scalable diffusion models with transformers[J]. Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023.
- [17] SALIMANS T, HO J. Progressive distillation for fast sampling of diffusion models[A]. 2022.
- [18] SONG Y, DHARIWAL P, CHEN M, et al. Consistency models[A]. 2023.
- [19] RADFORD A, KIM J W, HALLACY C, et al. Learning transferable visual models from natural language supervision[J]. International Conference on Machine Learning, 2021.